جزوه

Backend

بک‌اند: Python، TypeScript، C# و Java کنار هم

1. Concurrency in the backend · همروندی در بک‌اند: چند کار با هم، بدون اینکه یکی همه را بخواباند
2. API · HTTP API and routing و مسیرها: هر endpoint یک قرارداد است
3. Dependency injection and the composition root · تزریق وابستگی: برنامه نمی‌داند دیتابیس واقعی کدام است
4. Talking to a SQL database safely · کار با دیتابیس SQL: پارامتر، تراکنش، و کمترین دسترسی
5. Validating input · اعتبارسنجی ورودی: شکل درست، و هیچ فیلد اضافه‌ای
6. Error handling without leaking · مدیریت خطا: جزئیات در لاگ، نه در پاسخ
7. Authentication with JWT and roles · احراز هویت با JWT: توکن کوتاه، توکن تمدید، و نقش‌ها
8. Testing the real app with fake dependencies · تست: اپ واقعی، دیتابیس جعلی
9. Atomic state changes in MongoDB · تغییر وضعیت اتمیک در MongoDB: فقط یک نفر برنده می‌شود
10. Receiving webhooks safely · دریافت امن webhook: بدنه‌ی خام، امضا، تکرار
11. The outbox pattern · الگوی Outbox: پیام با داده ذخیره می‌شود، بعداً فرستاده می‌شود
12. Calling an LLM with tools · فراخوانی مدل زبانی با ابزار: حلقه، سقف، و ابزار فقط‌خواندنی

درس 1 · backend

Concurrency in the backend

همروندی در بک‌اند: چند کار با هم، بدون اینکه یکی همه را بخواباند

concurrency async workers

بعد از این درس می‌توانی

- درخواست‌های I/O مستقل را موازی بفرستی و نتیجه‌ها را یک‌جا جمع کنی (Task.WhenAll / Promise.all / asyncio.gather / CompletableFuture).
- بدانی چرا یک خطای تکی نباید کل پاسخ را بخواباند، و این را در کد واقعی ببینی.
- یک کارگر پس‌زمینه با حلقه و وقفه بنویسی که با خطا نمی‌میرد (BackgroundService / @Scheduled).

سناریو از پروژه‌ی واقعی

داشبورد Control Panel باید وضعیت چند پروژه را یک‌جا نشان بدهد. هر پروژه یک adapter دارد که روی پورت خودش جواب می‌دهد. اگر core-api آن‌ها را یکی‌یکی صدا بزند و هر کدام ۳ ثانیه طول بکشد، با ۹ پروژه صفحه ۲۷ ثانیه می‌ماند. اگر همه را همزمان بفرستد، کل صفحه فقط به اندازه کندترین adapter طول می‌کشد. اما اگر یکی از adapterها خاموش باشد، نباید کل داشبورد خطا بدهد. همین دو شرط، موازی بودن و تحمل خطا، قلب این درس است.

سؤال چرا adapterها را یکی‌یکی صدا نزنیم؟

پاسخ پس زمان کل می‌شود جمع زمان‌ها. با موازی کردن، می‌شود بیشترین زمان. چون کار ما صبر برای شبکه است، نه محاسبه، thread اضافه لازم نیست؛ فقط باید صبر را همزمان کنیم.

سؤال و اگر یکی از adapterها خاموش باشد؟

پاسخ همان‌جا که صدایش می‌زنیم خطا را می‌گیریم و به‌جایش healthy=false برمی‌گردانیم. پس عمل موازی هیچ‌وقت exception به بالا نمی‌دهد.

مفهوم‌ها

I/O-bound یا CPU-bound؟

اول بپرس کار از چه نوعی است. صبر برای شبکه یا دیسک (I/O) با async همزمان می‌شود و thread اضافه نمی‌خواهد. محاسبه سنگین (CPU) باید روی thread یا process جدا برود. در این درس هر چه از adapterها و outbox می‌خوانید I/O است.

Fan-out و Fan-in

Fan-out یعنی ساختن N کار مستقل و شروع همه با هم. Fan-in یعنی منتظر ماندن برای همه و جمع کردن نتیجه به همان ترتیب ورودی. ترتیب نتیجه همیشه ترتیب ورودی است، نه ترتیب تمام شدن.

خطا را کجا بگیریم؟

اگر هر کار تکی خطایش را خودش بگیرد و یک نتیجه‌ی «ناموفق» برگرداند، جمع‌کننده هیچ‌وقت خطا نمی‌بیند. اگر نگیرد، اولین خطا کل جمع‌کننده را می‌اندازد (در C# و Python بسته به API ممکن است بقیه‌ی کارها هم در پس‌زمینه ادامه دهند). کد واقعی ما راه اول را رفته است.

کارگر پس‌زمینه

یک حلقه: کار را انجام بده، خطا را لاگ کن اما حلقه را نکش، بعد مدتی صبر کن. صبر باید با درخواست توقف قطع شود، وگرنه خاموش کردن سرویس چند ثانیه دیر می‌شود.

مقایسه

چند کار I/O با هم، جمع نتیجه

C#	`await Task.WhenAll(tasks)`
TypeScript	`await Promise.all(promises)`
Python	`await asyncio.gather(*coros)`
Java	`CompletableFuture.allOf(...).join()`

در هر چهار، نتیجه به ترتیب ورودی است. فقط Python نیاز دارد * بزنید تا لیست به آرگومان باز شود.

کار «شروع شده» چیست؟

C#	`Task (داغ: با ساخت شروع می‌شود)`
TypeScript	`Promise (داغ)`
Python	`coroutine (تا سرد: await/gather/create_task اجرا نمی‌شود)`
Java	`CompletableFuture.supplyAsync (داغ)`

در Python اگر coroutine را فقط صدا بزنید و await نکنید، هیچ اتفاقی نمی‌افتد و فقط یک warning می‌گیرید.

خطا در یک کار	
C#	همه‌ی خطاها در) را می‌اندازد exception اولین WhenAll روی await می‌مانند) task.Exception.InnerExceptions
TypeScript	همه را می‌دهد allSettled رد می‌شود؛ reject با اولین Promise.all
Python	آن را به نتیجه تبدیل return_exceptions=True خطای اول را می‌اندازد؛ gather می‌کند
Java	می‌اندازد CompletionException یک join

اگر هر کار خودش خطایش را بگیرد (مثل fetchStatus)، این ردیف دیگر مهم نیست.

لغو و timeout	
C#	CancellationToken + CancelAfter
TypeScript	AbortSignal.timeout(ms)
Python	timeout= در httpx یا asyncio.timeout()
Java	HttpRequest.timeout(Duration)

در C# و TypeScript signal را باید خودتان به فراخوانی بدهید؛ فراموش کنید، timeout کار نمی‌کند.

کارگر پس‌زمینه	
C#	BackgroundService.ExecuteAsync
TypeScript	حلقه‌ی async + setTimeout/sleep
Python	asyncio.create_task در lifespan
Java	@Scheduled(fixedDelay)

Spring حلقه را برای شما می‌نویسد؛ در بقیه خودتان می‌نویسید.

صبر قابل‌توقف

C#	`Task.Delay(span, stoppingToken)`
TypeScript	`sleep(ms, undefined, { signal })`
Python	`asyncio.wait_for(stop.wait(), timeout)`
Java	`scheduler را Spring متوقف می‌کند`

C# با پرتاب OperationCanceledException از Delay خارج می‌شود؛ در حلقه‌ی ما همین استثنا از catch کنار گذاشته شده تا حلقه تمام شود.

thread واقعی

C#	`Task.Run (thread pool)`
TypeScript	`worker_threads (برای CPU)`
Python	`ThreadPoolExecutor / ProcessPoolExecutor (به‌خاطر GIL برای CPU process)`
Java	`ExecutorService / virtual threads (Java 21)`

برای I/O در Python و TypeScript تقریباً هیچ‌وقت thread لازم نیست.

کد

Fan-out به adapterها

C# کد اجراشده و تست‌شده در /examples examples/backend/csharp/Portfolio/Fanout.cs

```csharp
public async Task<IReadOnlyList<ProjectWithStatus>> ListWithStatusAsync(IEnumerable<Project> registry,
CancellationToken ct)
{
    // Each element starts immediately; WhenAll keeps the input order.
    return await Task.WhenAll(registry.Select(async p =>
        new ProjectWithStatus(p, await FetchStatusAsync(p, ct))));
}

// Never throws for an adapter problem: a down adapter becomes Healthy = false.
private async Task<AdapterStatus> FetchStatusAsync(Project p, CancellationToken ct)
{
    try
    {
        using var cts = CancellationTokenSource.CreateLinkedTokenSource(ct);
        cts.CancelAfter(TimeSpan.FromSeconds(3));

        using var res = await http.GetAsync($"{p.AdapterBaseUrl}/status", cts.Token);
        if (!res.IsSuccessStatusCode)
            return new(false, $"Adapter returned HTTP {(int)res.StatusCode}");

        return (await res.Content.ReadFromJsonAsync<AdapterStatus>(cts.Token))!;
    }
    catch (Exception) when (!ct.IsCancellationRequested)
    {
        // timeout, refused connection, bad JSON: all the same to the dashboard
        return new(false, "Adapter unreachable.");
    }
}
```

TypeScript از کد واقعی پروژه (کوتاه‌شده)

```typescript
async listWithStatus(): Promise<ProjectWithStatus[]> {
  const registry = await this.listRegistry();

  return Promise.all(
    registry.map(async (project) => ({
      ...project,
      status: await this.fetchStatus(project),
    })),
  );
}

private async fetchStatus(project: ProjectRegistryEntry): Promise<AdapterStatus> {
  try {
    const response = await this.requestAdapter(project, '/status');
    if (!response.ok) return { healthy: false, error: `Adapter returned HTTP ${response.status}` };
    return (await response.json()) as AdapterStatus;
  } catch (err) {
    this.logger.warn(`Adapter unreachable for ${project.key}: ${(err as Error).message}`);
    return { healthy: false, error: 'Adapter unreachable.' };
  }
}
```

Python کد اجراشده و تست‌شده در /examples

examples/backend/python/portfolio/fanout.py

```python
async def fetch_status(client: httpx.AsyncClient, p: Project) -> AdapterStatus:
    # Never raises for an adapter problem: a down adapter becomes healthy=False.
    try:
        res = await client.get(f"{p.adapter_base_url}/status", timeout=3.0)
        if res.status_code != 200:
            return AdapterStatus(healthy=False, error=f"Adapter returned HTTP {res.status_code}")
        return AdapterStatus(**res.json())
    except (httpx.HTTPError, ValueError):  # ValueError: bad JSON or a pydantic ValidationError
        return AdapterStatus(healthy=False, error="Adapter unreachable.")


async def list_with_status(
    client: httpx.AsyncClient, registry: list[Project]
) -> list[ProjectWithStatus]:
    # gather starts every call at once and keeps the input order.
    statuses = await asyncio.gather(*(fetch_status(client, p) for p in registry))
    return [ProjectWithStatus(project=p, status=s) for p, s in zip(registry, statuses)]
```

Java کد اجراشده و تست‌شده در /examples

examples/backend/java/src/main/java/portfolio/StatusFanout.java

```java
public List<ProjectWithStatus> listWithStatus(List<Project> registry) {
    // Start every call first, then join: the list keeps the input order.
    var futures = registry.stream()
            .map(p -> CompletableFuture.supplyAsync(() -> new ProjectWithStatus(p, fetchStatus(p)), executor))
            .toList();
    return futures.stream().map(CompletableFuture::join).toList();
}

// Never throws for an adapter problem: a down adapter becomes healthy = false.
AdapterStatus fetchStatus(Project p) {
    try {
        var req = HttpRequest.newBuilder(URI.create(p.adapterBaseUrl() + "/status"))
                .timeout(Duration.ofSeconds(3)).GET().build();
        var res = http.send(req, HttpResponse.BodyHandlers.ofString());
        if (res.statusCode() != 200)
            return new AdapterStatus(false, "Adapter returned HTTP " + res.statusCode());
        return mapper.readValue(res.body(), AdapterStatus.class);
    } catch (IOException e) { // includes a refused connection, a timeout and bad JSON
        return new AdapterStatus(false, "Adapter unreachable.");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // keep the interrupt flag
        return new AdapterStatus(false, "Interrupted.");
    }
}
```

کارگر پس‌زمینه‌ی Outbox

C# از کد واقعی پروژه (کوتاه‌شده)

```
public class OutboxWorker(OutboxDispatcher dispatcher, IOptionsMonitor<IntegrationOptions> options,
ILogger<OutboxWorker> logger) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested && options.CurrentValue.PollSeconds > 0)
        {
            try
            {
                await dispatcher.DispatchAsync(stoppingToken);
            }
            catch (Exception ex) when (ex is not OperationCanceledException)
            {
                logger.LogError(ex, "The outbox pass failed; trying again on the next one.");
            }
            await Task.Delay(TimeSpan.FromSeconds(options.CurrentValue.PollSeconds), stoppingToken);
        }
    }
}
```

TypeScript کد اجراشده و تست‌شده در /examples — examples/backend/typescript/src/worker.ts

```
export async function runWorker(
  dispatch: () => Promise<void>,
  pollSeconds: number,
  signal: AbortSignal,
  logger: { error(msg: string, err: unknown): void } = console,
): Promise<void> {
  while (!signal.aborted && pollSeconds > 0) {
    try {
      await dispatch();
    } catch (err) {
      logger.error('The outbox pass failed; trying again on the next one.', err);
    }
    // sleep rejects with AbortError when the signal fires; swallow it and let the loop end
    await sleep(pollSeconds * 1000, undefined, { signal }).catch(() => undefined);
  }
}
```

Python کد اجراشده و تست‌شده در /examples

examples/backend/python/portfolio/worker.py

```python
async def run_worker(
    dispatch: Callable[[], Awaitable[None]],
    poll_seconds: float,
    stop: asyncio.Event,
) -> None:
    while not stop.is_set() and poll_seconds > 0:
        try:
            await dispatch()
        except Exception:  # CancelledError is a BaseException, so it passes through
            logger.exception("The outbox pass failed; trying again on the next one.")
        try:
            # wakes up early when stop is set, so shutdown does not wait out the delay
            await asyncio.wait_for(stop.wait(), timeout=poll_seconds)
        except asyncio.TimeoutError:
            pass
```

Java کد اجراشده و تست‌شده در /examples

examples/backend/java/src/main/java/portfolio/OutboxWorker.java

```java
// Registered as a bean in PortfolioConfig; @EnableScheduling there turns @Scheduled on.
public class OutboxWorker {
    private static final Logger log = LoggerFactory.getLogger(OutboxWorker.class);
    private final Runnable dispatcher;

    public OutboxWorker(Runnable dispatcher) {
        this.dispatcher = dispatcher;
    }

    // fixedDelay waits AFTER a pass finishes (like the C# loop); fixedRate would not.
    @Scheduled(fixedDelayString = "PT${integrations.poll-seconds:10}S")
    public void run() {
        try {
            dispatcher.run();
        } catch (Exception e) {
            log.error("The outbox pass failed; trying again on the next one.", e);
        }
    }
}
```

اشتباه‌های رایج

TypeScript **await داخل for روی یک آرایه از adapterها**

چرا: هر await قبل از شروع بعدی تمام می‌شود، پس کار پشت‌سرهم اجرا می‌شود و موازی نیست.

درست: اول map بزن تا همه‌ی promiseها ساخته شوند، بعد یک بار Promise.all.

Python **صدا زدن fetch_status(...) بدون await یا gather**

چرا: coroutine سرد است: ساخته می‌شود اما اجرا نمی‌شود، فقط RuntimeWarning: coroutine was never awaited.

درست: آن را به gather بدهید یا await کنید.

Python **استفاده از requests.get داخل async def**

چرا: requests همزمان (blocking) است و کل event loop را می‌خواباند؛ gather هیچ فایده‌ای نمی‌کند.

درست: از httpx.AsyncClient یا aiohttp استفاده کنید.

Java **CompletableFuture.supplyAsync بدون executor**

چرا: از ForkJoinPool.commonPool استفاده می‌کند که برای کار CPU است و تعداد threadهایش کم است؛ I/O کند آن را اشباع می‌کند.

درست: executor خودتان بدهید (مثلاً virtual threads در Java 21).

Java **catch (InterruptedException e) {} خالی**

چرا: علامت interrupt گم می‌شود و کسی که می‌خواهد thread را متوقف کند موفق نمی‌شود.

درست: Thread.currentThread().interrupt() را دوباره صدا بزنید.

C# **catch (Exception) بدون when(...) در حلقه‌ی worker**

چرا: OperationCanceledException هم گرفته می‌شود و سرویس هنگام خاموش شدن تا Task.Delay بعدی بیدار نمی‌شود.

درست: when (ex is not OperationCanceledException) را اضافه کنید.

همه **Promise.all / WhenAll / gather بدون تحمل خطا روی چند منبع**

چرا: اولین خطا کل نتیجه را می‌اندازد و داشبورد با خاموش بودن یک adapter کامل خراب می‌شود.

درست: یا خطا را داخل هر کار بگیرید (مثل fetchStatus)، یا return_exceptions=True / allSettled بزنید.

این کد کجای پروژه‌هاست

`ControlPanel/services/core-api/src/projects/projects.service.ts:43`

Promise.all روی adapterها (fan-out/fan-in)

`ControlPanel/services/core-api/src/projects/projects.service.ts:124`

fetchStatus: خطا را می‌گیرد و healthy=false برمی‌گرداند

`ControlPanel/services/core-api/src/assistant/assistant.service.ts:95`

Promise.all برای پاسخ هم‌زمان ابزارهای مدل

`ClinicHub/src/ClinicHub.Api/Integrations/OutboxDispatcher.cs:245`

OutboxWorker: حلقه‌ی BackgroundService با catch فیلتری

`ClinicHub/tests/ClinicHub.Tests/EncounterTests.cs:177`

Task.WhenAll در تست برای ذخیره‌های همزمان

تمرین

▼ 1. در نسخه‌ی TypeScript یک adapter را عمداً خاموش کنید. چه چیزی از fetchStatus برمی‌گردد و چرا Promise.all تمام می‌شود؟

راهنما: به catch در fetchStatus نگاه کنید.

پاسخ: { healthy: false, error: 'Adapter unreachable' }. چون fetchStatus خودش خطا را می‌گیرد، promise هیچ‌وقت reject نمی‌شود و Promise.all همه را تمام شده می‌بیند.

▼ 2. catch را از fetchStatus حذف کنید و به‌جایش در listWithStatus از Promise.allSettled استفاده کنید. نتیجه‌ی هر آیتم چه شکلی می‌شود و کجا باید آن را به AdapterStatus تبدیل کنید؟

پاسخ: هر آیتم { status: 'fulfilled', value } یا { status: 'rejected', reason } است. تبدیل rejected به { healthy: false, error } باید در listWithStatus انجام شود، یعنی همان منطقی که قبلاً در catch بود فقط جابه‌جا شده.

▼ 3. در نسخه‌ی Python به‌جای gather از یک for ساده با await استفاده کنید و زمان را برای ۵ adapter که هر کدام ۱ ثانیه جواب می‌دهند اندازه بگیرید. با gather چقدر می‌شود؟

راهنما: time.perf_counter() و asyncio.sleep(1) به‌جای شبکه.

پاسخ: با for حدود ۵ ثانیه، با gather حدود ۱ ثانیه.

▼ 4. در Java @Scheduled(fixedDelay) را با fixedRate عوض کنید. اگر dispatcher.dispatch() ۱۰ ثانیه طول بکشد و poll هر ۵ ثانیه باشد چه تفاوتی می‌بینید؟

پاسخ: fixedRate بعد از شروع هر اجرا حساب می‌کند. اگر اجرا طولانی شود، اجرای بعدی بلافاصله بعد از تمام شدن شروع می‌شود (روی یک thread اجرا همپوشانی ندارد). fixedDelay همیشه بعد از پایان ۵ ثانیه صبر می‌کند، که همان رفتار حلقه‌ی C# است.

جمله‌ی مصاحبه

I run independent calls in parallel, and I make each one handle its own failure, so one slow or broken service never takes the whole dashboard down.

درس 2 · backend

HTTP API and routing

API و مسیرها: هر endpoint یک قرارداد است

http routing status codes

بعد از این درس می‌توانی

- یک endpoint از نوع GET و یک POST در هر چهار زبان بنویسی و ببینی که ساختار همه یکی است: مسیر، ورودی، خروجی، کد وضعیت.
- کد وضعیت درست را انتخاب کنی: 200، 201 با 409 ،404 ،Location.
- بدانی «سالم نبودن» یک پاسخ است و نه یک خطا، مثل /status در adapterهای Control Panel.

سناریو از پروژه‌ی واقعی

هر adapter در Control Panel فقط یک قرارداد دارد: GET /status که { healthy, summary, metrics } برمی‌گرداند. core-api هیچ‌چیز دیگری از پروژه‌ها نمی‌داند. وقتی دیتابیس LedgerDashboard در دسترس نیست، adapter باز هم 200 می‌دهد، ولی با healthy=false، چون داشبورد باید بتواند این وضعیت را نشان بدهد. در این درس همان /status را به‌اضافه‌ی GET /projects/{key} و POST /projects در چهار زبان می‌سازیم.

سؤال چرا /status وقتی دیتابیس خاموش است 500 برنمی‌گرداند؟

پاسخ چون سؤال «آیا سالمی؟» جواب دارد: «نه». 500 یعنی خود endpoint خراب شده. داشبورد با 200 و healthy=false می‌تواند کارت قرمز نشان بدهد، اما با 500 فقط می‌داند که چیزی نامعلوم شکسته است.

سؤال چرا 201 و Location در POST؟

پاسخ 201 می‌گوید چیزی ساخته شد و Location می‌گوید کجا. کلاینت لازم نیست آدرس را حدس بزند.

مفهوم‌ها

مسیر، متد، پارامتر

در هر چهار فریم‌ورک یک endpoint از سه چیز ساخته می‌شود: متد HTTP، الگوی مسیر با پارامتر ({key} یا :key)، و تابعی که پاسخ را برمی‌گرداند. فقط شکل نوشتن فرق دارد: C# و Python و TypeScript تابع را مستقیم به مسیر وصل می‌کنند (Minimal API، FastAPI، Express)، Java آن را روی متد یک کلاس با annotation می‌گذارد.

برگرداندن JSON

در C# و FastAPI و Spring یک شیء برمی‌گردانید و فریم‌ورک آن را JSON می‌کند. در Express خودتان res.json() را صدا می‌زنید. در C# برای کد وضعیت از Results.* استفاده می‌کنید، در Spring از ResponseEntity یا یک exception.

کد وضعیت را عمداً انتخاب کن

200 برای خواندن موفق، 201 + Location برای ساختن، 400 برای ورودی بد، 401 برای «وارد نشده‌ای»، 403 برای «اجازه نداری»، 404 برای «پیدا نشد»، 409 برای «با وضعیت فعلی تداخل دارد»، و 500 فقط برای «خود من خراب شدم».

Express 4 و async

Express 4 promise رد شده‌ی یک handler را نمی‌گیرد. اگر await داخل handler خطا بدهد و خودتان آن را به next(err) ندهید، درخواست بی‌پاسخ می‌ماند. Express 5 این را درست کرده است؛ کد ما برای Express 4 است و try/catch دارد.

مقایسه

تعریف یک GET

C#	`app.MapGet("/projects/{key}", (string key) => ...)`
TypeScript	`app.get('/projects/:key', (req, res) => ...)`
Python	`@app.get("/projects/{key}")`
Java	`@GetMapping("/projects/{key}")`

Express پارامتر را با : می‌نویسد و از req.params می‌خواند؛ بقیه آن را مستقیم به تابع می‌دهند.

پارامتر مسیر

C#	`(string key)`
TypeScript	`req.params.key`
Python	`def get_project(key: str)`
Java	`@PathVariable String key`

در C# و Python نام پارامتر تابع باید با {key} یکی باشد؛ در Java با annotation وصل می‌شود.

404 با پیام

C#	`Results.NotFound(new { error = ... })`
TypeScript	`res.status(404).json({ error })`
Python	`raise HTTPException(404, ...)`
Java	`throw new ResponseStatusException(NOT_FOUND, ...)`

Python و Java با پرتاب exception جواب می‌دهند؛ C# و Express مقدار برمی‌گردانند.

201 با Location

C#	`Results.Created($"/projects/{key}", row)`
TypeScript	`res.status(201).location(url).json(row)`
Python	`JSONResponse(..., status_code=201, headers={"Location": url})`
Java	`ResponseEntity.created(URI.create(url)).body(row)`

FastAPI کمکی برای Location ندارد؛ هدر را خودتان می‌گذارید.

بدنه‌ی JSON درخواست

C#	`(CreateProject body)`
TypeScript	`app.use(express.json()); req.body`
Python	`def create(body: CreateProject)`
Java	`@RequestBody CreateProject body`

Express بدون express.json() اصلاً بدنه را نمی‌خواند و req.body خالی است.

کد

status/ و projects/

C# کد اجراشده و تست‌شده در /examples examples/backend/csharp/Portfolio/PortfolioApp.cs

```
app.MapGet("/status", async (IStatusQueries q) =>
{
    try
    {
        var s = await q.StatusAsync();
        var m = new { sent = s.Sent ?? 0, interviewing = s.Interviewing ?? 0, followupsDue = s.FollowupsDue ??
0 };
        return Results.Ok(new { healthy = true, summary = $"{m.sent} sent, {m.interviewing} interviewing,
{m.followupsDue} follow-up(s) due", metrics = m });
    }
    catch (Exception)
    {
        // Unhealthy is an answer, not an error: the dashboard shows it.
        return Results.Ok(new { healthy = false, error = "Database unreachable." });
    }
});

app.MapGet("/projects/{key}", async (string key, IProjectRepository repo) =>
    await repo.FindAsync(key) is { } row
        ? Results.Ok(row)
        : Results.NotFound(new { error = $"Unknown project: {key}" }));

app.MapPost("/projects", async (CreateProject body, IProjectRepository repo) =>
{
    var problems = new List<ValidationResult>();
    if (!Validator.TryValidateObject(body, new ValidationContext(body), problems, validateAllProperties: true))
        return Results.BadRequest(new
        {
            error = "Invalid request.",
            errors = problems.ToDictionary(p => JsonNamingPolicy.CamelCase.ConvertName(p.MemberNames.First()),
p => p.ErrorMessage),
        });
    if (await repo.FindAsync(body.Key) is not null)
        return Results.Conflict(new { error = $"Project {body.Key} already exists." });
    var row = new ProjectRow(body.Key, body.DisplayName, body.AdapterBaseUrl);
    await repo.AddAsync(row);
    return Results.Created($"/projects/{row.Key}", row);
}).AddEndpointFilter(RequireRole("Admin"));
```

examples/backend/typescript/src/app.ts کد اجراشده و تست‌شده در /examples **TypeScript**

```
app.get('/status', async (_req, res) => {
  try {
    const s = await queries.status();
    const m = { sent: s.sent ?? 0, interviewing: s.interviewing ?? 0, followupsDue: s.followupsDue ?? 0 };
    const summary = `${m.sent} sent, ${m.interviewing} interviewing, ${m.followupsDue} follow-up(s) due`;
    res.json({ healthy: true, summary, metrics: m });
  } catch {
    // Unhealthy is an answer, not an error: the dashboard shows it.
    res.json({ healthy: false, error: 'Database unreachable.' });
  }
});

// Express 4 does not catch a rejected promise from a handler: pass errors to next() yourself.
app.get('/projects/:key', async (req, res, next) => {
  try {
    const row = await projects.find(req.params.key);
    if (!row) throw new HttpError(404, `Unknown project: ${req.params.key}`);
    res.json(row);
  } catch (err) {
    next(err);
  }
});

app.post('/projects', requireRole('Admin'), async (req, res, next) => {
  try {
    const parsed = CreateProject.safeParse(req.body);
    if (!parsed.success) {
      const errors = Object.fromEntries(
        // an unknown field has an empty path and names the field in `keys`
        parsed.error.issues.map((i) => [i.path.join('.') || ('keys' in i ? i.keys.join(',') : 'body'),
i.message]),
      );
      return res.status(400).json({ error: 'Invalid request.', errors });
    }
    if (await projects.find(parsed.data.key)) throw new HttpError(409, `Project ${parsed.data.key} already
exists.`);
    await projects.add(parsed.data);
    res.status(201).location(`/projects/${parsed.data.key}`).json(parsed.data);
  } catch (err) {
    next(err);
  }
});
```

Python کد اجراشده و تست‌شده در /examples examples/backend/python/portfolio/app.py

```python
@app.get("/status")
def status():
    try:
        s = queries.status()
    except Exception:
        # Unhealthy is an answer, not an error: the dashboard shows it.
        return {"healthy": False, "error": "Database unreachable."}
    m = {k: v or 0 for k, v in s.model_dump().items()}
    summary = f"{m['sent']} sent, {m['interviewing']} interviewing, {m['followups_due']} follow-up(s) due"
    return {"healthy": True, "summary": summary, "metrics": m}

@app.get("/projects/{key}")
def get_project(key: str):
    row = projects.find(key)
    if row is None:
        raise HTTPException(404, f"Unknown project: {key}")
    return row

@app.post("/projects", status_code=201, dependencies=[Depends(require_role("Admin"))])
def create_project(body: CreateProject):
    if projects.find(body.key):
        raise HTTPException(409, f"Project {body.key} already exists.")
    row = ProjectRow(key=body.key, display_name=body.display_name, adapter_base_url=str(body.adapter_base_url))
    projects.add(row)
    return JSONResponse(row.model_dump(), status_code=201, headers={"Location": f"/projects/{row.key}"})
```

Java کد اجراشده و تست‌شده در examples/

examples/backend/java/src/main/java/portfolio/PortfolioController.java

```java
@RestController
public class PortfolioController {
    private final StatusQueries queries;
    private final ProjectRepository projects;

    // Constructor injection: Spring passes the beans in; a test can register fakes instead.
    public PortfolioController(StatusQueries queries, ProjectRepository projects) {
        this.queries = queries;
        this.projects = projects;
    }

    @GetMapping("/status")
    public Map<String, Object> status() {
        try {
            var s = queries.status();
            int sent = s.sent() == null ? 0 : s.sent();
            int interviewing = s.interviewing() == null ? 0 : s.interviewing();
            int followupsDue = s.followupsDue() == null ? 0 : s.followupsDue();
            return Map.of("healthy", true,
                    "summary", sent + " sent, " + interviewing + " interviewing, " + followupsDue + " follow-
up(s) due",
                    "metrics", Map.of("sent", sent, "interviewing", interviewing, "followupsDue",
followupsDue));
        } catch (Exception e) {
            // Unhealthy is an answer, not an error: the dashboard shows it.
            return Map.of("healthy", false, "error", "Database unreachable.");
        }
    }

    @GetMapping("/projects/{key}")
    public ProjectRow get(@PathVariable String key) {
        return projects.find(key)
                .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Unknown project: " +
key));
    }

    @PostMapping("/projects")
    @RequireRole("Admin")
    public ResponseEntity<ProjectRow> create(@Valid @RequestBody CreateProject body) {
        if (projects.find(body.key()).isPresent())
            throw new ResponseStatusException(HttpStatus.CONFLICT, "Project " + body.key() + " already
exists.");
        var row = new ProjectRow(body.key(), body.displayName(), body.adapterBaseUrl());
        projects.add(row);
        return ResponseEntity.created(URI.create("/projects/" + row.key())).body(row);
    }
}
```

اشتباه‌های رایج

TypeScript **handler async در Express 4 بدون try/catch و next(err)**

چرا: Express 4 promise رد شده را نمی‌بیند؛ درخواست تا timeout معلق می‌ماند و خطا فقط در لاگ unhandledRejection دیده می‌شود.

درست: try/catch و next(err)، یا Express 5 که این را خودش انجام می‌دهد.

همه **برگرداندن 500 برای «پیدا نشد» یا «سالم نیست»**

چرا: کلاینت و مانیتورینگ نمی‌توانند خرابی واقعی را از پاسخ عادی جدا کنند.

درست: 404 برای نبودن، 200 با healthy=false برای ناسالم بودن، 500 فقط برای باگ.

Java **برگرداندن null از متد کنترلر برای «پیدا نشد»**

چرا: Spring یک 200 با بدنه‌ی خالی می‌فرستد.

درست: Optional.orElseThrow با ResponseStatusException(NOT_FOUND).

Python **فراموش کردن status_code=201 در POST**

چرا: FastAPI به‌طور پیش‌فرض 200 می‌دهد، حتی برای ساختن.

درست: status_code=201 روی decorator، یا JSONResponse با کد.

این کد کجای پروژه‌هاست

`ControlPanel/services/adapters/ledgerdash-adapter/src/app.ts:51`

GET /status: سالم نبودن هم 200 با healthy=false است

`ControlPanel/services/core-api/src/projects/projects.controller.ts:27`

POST /projects در NestJS با @Roles('Admin')

`ClinicReport/src/ClinicReport.Api/Endpoints/AuthEndpoints.cs:47`

C#: MapPost("/login", ...) در Minimal API

تمرین

▼ 1. یک endpoint جدید GET /projects بنویس که همه‌ی پروژه‌ها را برگرداند، در زبانی که کمتر بلدی. چه چیزی را باید به interface مخزن اضافه کنی؟

راهنما: به ProjectRepository یک متد list اضافه کن و پیاده‌سازی in-memory را هم کامل کن.

پاسخ: یک متد list/all در ProjectRepository، پیاده‌سازی آن در InMemoryProjects و SqliteProjects، و یک route که نتیجه را برمی‌گرداند. هیچ چیز دیگری در route لازم نیست.

▼ 2. اگر دو بار پشت سر هم POST /projects با همان key بفرستی، بار دوم چه کدی برمی‌گردد و چرا 409 و نه 400؟

پاسخ: 409. ورودی از نظر شکل درست است (پس 400 نیست)، اما با وضعیت فعلی سرور تداخل دارد: پروژه‌ای با این key وجود دارد.

▼ 3. در نسخه‌ی Express، try/catch را از GET /projects/:key بردار و یک خطا در find بینداز. با Express 4 چه می‌بینی؟

پاسخ: درخواست جواب نمی‌گیرد (تا timeout کلاینت) و Node یک unhandledRejection لاگ می‌کند. با next(err) خطا به error handler می‌رسد و 500 برمی‌گردد.

جمله‌ی مصاحبه

I treat every endpoint as a contract: a clear route, the right status code, and an unhealthy dependency reported as data, not as a 500.

درس 3 · backend

Dependency injection and the composition root

تزریق وابستگی: برنامه نمی‌داند دیتابیس واقعی کدام است

dependency injection | ports | composition root

بعد از این درس می‌توانی

- وابستگی‌ها را به‌صورت interface (port) تعریف کنی و پیاده‌سازی را از بیرون بدهی.
- تفاوت DI دستی (پارامتر factory در Python و TypeScript) و container (ASP.NET و Spring) را ببینی.
- یک جای واحد به نام composition root داشته باشی که تنها جایی است که پیاده‌سازی‌های واقعی را می‌شناسد.

سناریو از پروژه‌ی واقعی

ledgerdash-adapter در Control Panel یک تابع createApp(queries) دارد. خود اپ نمی‌داند queries از SQL Server می‌خواند یا از یک شیء جعلی در تست. main.ts تنها جایی است که اتصال واقعی mssql را می‌سازد و به createApp می‌دهد. نتیجه: تست‌ها یک اپ Express واقعی را اجرا می‌کنند و فقط دیتابیس جعلی است. همین الگو را در چهار زبان پیاده می‌کنیم.

سؤال در C# که container داریم، چرا باز هم Build(queries, projects, tokens) پارامتر می‌گیرد؟

پاسخ تا تست بتواند بدون container سازی و بدون override کردن سرویس‌ها، پیاده‌سازی جعلی را مستقیم بدهد. Build آن‌ها را در container ثبت می‌کند و endpointها مثل همیشه از container می‌گیرند.

سؤال Spring چطور؟ آنجا new نمی‌کنیم.

پاسخ در Spring کنترلر از constructor وابستگی می‌گیرد و beanها در یک کلاس @Configuration ساخته می‌شوند. تست با profile جدا beanهای جعلی را ثبت می‌کند. شکلش فرق دارد، ایده یکی است.

مفهوم‌ها

Port یعنی آنچه اپ لازم دارد، نه آنچه دیتابیس دارد

StatusQueries فقط status() دارد و ProjectRepository فقط find و add. اپ به همین‌ها وابسته است. SQL Server، SQLite یا یک dictionary در حافظه هر سه می‌توانند این قرارداد را پیاده کنند.

DI دستی در برابر container

در Python و TypeScript وابستگی‌ها پارامتر factory هستند (create_app(queries, projects, tokens)). FastAPI یک سیستم Depends هم دارد، اما برای این اندازه لازم نیست. در ASP.NET و Spring یک container ساخت اشیاء و طول عمرشان را مدیریت می‌کند.

Composition root

Program.cs، main.ts، main.py و کلاس Real در PortfolioApp.java: تنها جایی که SQLite واقعی، secretها و تنظیمات خوانده می‌شوند. هیچ جای دیگری new SqliteProjects نمی‌زند.

Interface در Python

Python به‌جای interface از typing.Protocol استفاده می‌کند: هر کلاسی که متدهای لازم را داشته باشد قبول است، بدون اینکه چیزی را implement کند (structural typing). TypeScript هم همین‌طور است. C# و Java باید صریحاً implements بنویسند.

مقایسه

تعریف port	
C#	`interface IProjectRepository`
TypeScript	`interface ProjectRepository`
Python	`class ProjectRepository(Protocol)`
Java	`interface ProjectRepository`

Python و TypeScript ساختاری‌اند: لازم نیست کلاس بگوید implements.

گرفتن وابستگی	
C#	`پارامتر endpoint یا primary constructor`
TypeScript	`پارامتر createApp`
Python	`پارامتر create_app`
Java	`کنترلر constructor`

C# می‌تواند وابستگی را مستقیم به پارامتر lambda بدهد؛ Spring فقط از constructor (یا field).

ثبت پیاده‌سازی	
C#	`builder.Services.AddSingleton(projects)`
TypeScript	`createApp(queries, projects, tokens)`
Python	`create_app(queries, projects, tokens)`
Java	`@Bean ProjectRepository projects()`

در Python و TypeScript «container» همان فراخوانی تابع است.

جایگزینی در تست	
C#	`Build(fakes, b => b.WebHost.UseTestServer())`
TypeScript	`createApp(fakeQueries(), ...)`
Python	`create_app(FakeQueries(), ...)`
Java	`@ActiveProfiles("test") + @TestConfiguration`

Spring به profile نیاز دارد تا beanهای واقعی (که secret می‌خواهند) ساخته نشوند.

طول عمر	
C#	`Singleton / Scoped / Transient`
TypeScript	`خودتان (ماژول = singleton)`
Python	`FastAPI در Depends خودتان؛ یا`
Java	`@Scope("prototype") پیش‌فرض؛ singleton`

Scoped در ASP.NET یعنی یکی برای هر درخواست؛ DbContext باید Scoped باشد.

کد

portها

C# کد اجراشده و تست‌شده در /examples — examples/backend/csharp/Portfolio/Models.cs

```
public interface IStatusQueries
{
    Task<StatusCounts> StatusAsync();
}

public interface IProjectRepository
{
    Task<ProjectRow?> FindAsync(string key);
    Task AddAsync(ProjectRow row);
}
```

TypeScript کد اجراشده و تست‌شده در /examples — examples/backend/typescript/src/models.ts

```
export interface StatusQueries {
  status(): Promise<StatusCounts>;
}

export interface ProjectRepository {
  find(key: string): Promise<ProjectRow | undefined>;
  add(row: ProjectRow): Promise<void>;
}
```

Python کد اجراشده و تست‌شده در /examples — examples/backend/python/portfolio/models.py

```
class StatusQueries(Protocol):
    def status(self) -> StatusCounts: ...


class ProjectRepository(Protocol):
    def find(self, key: str) -> ProjectRow | None: ...
    def add(self, row: ProjectRow) -> None: ...
```

Java کد اجراشده و تست‌شده در /examples — examples/backend/java/src/main/java/portfolio/Ports.java

```
public interface StatusQueries {
    StatusCounts status() throws Exception;
}

public interface ProjectRepository {
    Optional<ProjectRow> find(String key);
    void add(ProjectRow row);
}
```

ساختن اپ با وابستگی‌های داده‌شده

C# کد اجراشده و تست‌شده در /examples — examples/backend/csharp/Portfolio/PortfolioApp.cs

```
public static WebApplication Build(IStatusQueries queries, IProjectRepository projects, TokenService tokens,
    Action<WebApplicationBuilder>? configure = null)
{
    // Everything the app needs comes in as an argument, so a test can pass fakes.
    var builder = WebApplication.CreateBuilder();
    builder.Services.AddSingleton(queries).AddSingleton(projects).AddSingleton(tokens);
    // Body-binding failures (bad JSON, unknown fields) reach our error handler instead of a bare 400.
    builder.Services.Configure<RouteHandlerOptions>(o => o.ThrowOnBadRequest = true);
    configure?.Invoke(builder);
    var app = builder.Build();
```

TypeScript کد اجراشده و تست‌شده در /examples — examples/backend/typescript/src/app.ts

```
export function createApp(queries: StatusQueries, projects: ProjectRepository, tokens: TokenService) {
  // Everything the app needs comes in as an argument, so a test can pass fakes.
  const app = express();
  app.use(express.json());
```

Python کد اجراشده و تست‌شده در /examples — examples/backend/python/portfolio/app.py

```
def create_app(queries: StatusQueries, projects: ProjectRepository, tokens: TokenService) -> FastAPI:
    # Everything the app needs comes in as an argument, so a test can pass fakes.
    app = FastAPI()
```

Java کد اجراشده و تست‌شده در examples/

examples/backend/java/src/main/java/portfolio/PortfolioApp.java

```java
// Composition root: the only place that knows which real implementations are used.
// Tests run with the "test" profile and register fakes instead.
@Configuration
@Profile("!test")
@EnableScheduling
static class Real {
    private final Config.Settings settings = Config.load(System.getenv());

    @Bean
    ProjectRepository projects() throws SQLException {
        return new SqliteProjects(DriverManager.getConnection("jdbc:sqlite:" + settings.dbPath()));
    }

    @Bean
    StatusQueries queries() {
        return () -> new StatusCounts(0, 0, 0);
    }

    @Bean
    TokenService tokens() {
        return new TokenService(settings.jwtSecret(), settings.jwtRefreshSecret(), id -> null);
    }

    @Bean
    Webhook.Settings webhook() {
        return new Webhook.Settings(List.of(System.getenv().getOrDefault("WEBHOOK_SECRET", "")),
                event -> {}, new HashSet<>(), () -> System.currentTimeMillis() / 1000);
    }
}
```

اشتباه‌های رایج

همه **new کردن دیتابیس داخل handler یا سرویس**

چرا: تست مجبور است دیتابیس واقعی داشته باشد و هیچ جای دیگری نمی‌تواند پیاده‌سازی را عوض کند.

درست: وابستگی را از constructor یا پارامتر factory بگیر و فقط در composition root بساز.

C# **ثبت DbContext یا سرویسی که DbContext دارد به‌صورت Singleton**

چرا: DbContext thread-safe نیست و بین درخواست‌ها مشترک می‌شود.

درست: Scoped (AddDbContext) خودش Scoped ثبت می‌کند).

Java **@Autowired روی field به‌جای constructor**

چرا: کلاس بدون Spring قابل ساختن نیست و وابستگی‌ها پنهان می‌مانند؛ field می‌تواند null باشد.

درست: constructor injection با فیلدهای final.

Python **import کردن اتصال دیتابیس سطح ماژول در فایل app**

چرا: صرفاً import کردن ماژول دیتابیس را باز می‌کند و تست نمی‌تواند آن را عوض کند.

درست: اتصال را در main.py بساز و به create_app بده.

این کد کجای پروژه‌هاست

`ControlPanel/services/adapters/ledgerdash-adapter/src/app.ts:48`

createApp(queries): اپ با port داده‌شده ساخته می‌شود

`ControlPanel/services/adapters/ledgerdash-adapter/src/app.spec.ts:24`

fakeQueries: پیاده‌سازی جعلی همان port

`ClinicHub/src/ClinicHub.Api/Integrations/OutboxDispatcher.cs:18`

primary constructor در C#: وابستگی‌ها از container

تمرین

▼ 1. یک پیاده‌سازی دوم از StatusQueries بنویس که همیشه یک عدد ثابت برگرداند، و آن را فقط در composition root عوض کن. چند فایل عوض شد؟

پاسخ: فقط یک فایل جدید (پیاده‌سازی) و composition root. app/route هیچ تغییری نمی‌کند؛ این نشانه‌ی درستی جداسازی است.

▼ 2. در Spring چرا کلاس Real با @Profile("!test") علامت خورده؟ اگر آن را برداری چه می‌شود؟

پاسخ: Real در ساخته شدن Config.load(System.getenv()) را صدا می‌زند که بدون JWT_SECRET خطا می‌دهد. بدون profile، context تست بالا نمی‌آید؛ تازه beanهای تکراری هم با beanهای جعلی تداخل می‌کنند.

▼ 3. در C# یک سرویس Scoped را از یک Singleton بگیر. در Development چه خطایی می‌گیری و چرا؟

پاسخ: ASP.NET در Development scope validation دارد و خطای «Cannot consume scoped service from singleton» می‌دهد. Singleton عمر بیشتری دارد و سرویس Scoped را بعد از پایان درخواست نگه می‌دارد (captive dependency).

جمله‌ی مصاحبه

My handlers depend on small interfaces; only the composition root knows the real database, so tests run the real app with fakes.

درس 4 · backend

Talking to a SQL database safely

کار با دیتابیس SQL: پارامتر، تراکنش، و کمترین دسترسی

sql | parameters | least privilege

بعد از این درس می‌توانی

- در هر چهار زبان کوئری پارامتری بنویسی و ببینی چرا تلاش برای SQL injection به یک متن ساده تبدیل می‌شود.
- منابع دیتابیس (connection، statement، reader) را درست ببندی: free، try-with-resources، with، using.
- بفهمی چرا ledgerdash-adapter با لاگینی کار می‌کند که فقط viewها را می‌بیند.

سناریو از پروژه‌ی واقعی

ledgerdash-adapter به دیتابیس LedgerDashboard وصل می‌شود، ولی با لاگین ledger_reader که هیچ جدولی را نمی‌بیند و فقط viewهای گزارش را می‌خواند. حتی اگر کسی راهی برای فرستادن SQL پیدا کند، چیزی برای خراب کردن ندارد. core-api هم هر مقداری را با input('key', ...). می‌فرستد، نه با چسباندن رشته. در این درس مخزن پروژه‌ها را روی SQLite با همین دو اصل می‌سازیم.

سؤال اگر key را با رشته در SQL بچسبانم و قبلاً اعتبارسنجی کرده باشم، باز هم مشکل دارد؟

پاسخ بله. اعتبارسنجی یک لایه است، ولی پارامتر تنها چیزی است که تضمین می‌کند مقدار هرگز به‌عنوان SQL خوانده نشود. یک روز کسی قاعده‌ی اعتبارسنجی را شل می‌کند و آن روز injection برمی‌گردد.

سؤال چرا لاگین فقط‌خواندنی، وقتی کد ما فقط SELECT می‌زند؟

پاسخ چون کد امروز را تضمین می‌کند، نه کد فردا یا باگ فردا را. دسترسی در دیتابیس مرز آخر است.

مفهوم‌ها

پارامتر یعنی مقدار جدا از متن SQL

با پارامتر، متن کوئری و مقدار جدا به درایور می‌روند. درایور (یا خود دیتابیس) مقدار را هرگز parse نمی‌کند. در تست هر چهار زبان، کلیدی مثل x'; DROP TABLE projects; -- به‌عنوان یک رشته‌ی عادی ذخیره و پیدا می‌شود.

نماد پارامتر در هر درایور

sqlite3 در Python و sql.js در TypeScript و JDBC از ? استفاده می‌کنند، Microsoft.Data.Sqlite از $name یا @،name mssql در Node از @name با .input()، و EF Core خودش پارامتر می‌سازد.

بستن منابع

connection و command و reader منابع غیر حافظه‌ای دارند. C#: using/await using. Java: try-with-resources. Python: with. sql.js: stmt.free()، چون حافظه‌ی WebAssembly را GC آزاد نمی‌کند.

تراکنش

Python: with conn: (commit در پایان، rollback در exception). C#: BeginTransaction و Commit؛ dispose بدون Commit یعنی rollback. Java: setAutoCommit(false)، commit، rollback. درس Outbox از این استفاده می‌کند.

کمترین دسترسی

لاگین برنامه فقط به چیزی دسترسی داشته باشد که لازم دارد: یک adapter گزارش فقط SELECT روی viewها. اسکریپت ساخت کاربر در خود ریپو باشد تا قابل بازبینی باشد.

مقایسه

کوئری پارامتری	
C#	`cmd.Parameters.AddWithValue("$key", key)`
TypeScript	`stmt.bind([key])`
Python	`conn.execute(sql, (key,))`
Java	`ps.setString(1, key)`

Python یک tuple می‌خواهد: (key,) با ویرگول، نه (key).

بستن منبع	
C#	`await using var cmd = ...`
TypeScript	`try { ... } finally { stmt.free() }`
Python	`with conn:`
Java	`try (var ps = ...) { }`

with در sqlite3 پایتون تراکنش را می‌بندد، نه اتصال را.

خواندن یک سطر	
C#	`if (await reader.ReadAsync()) reader.GetString(0)`
TypeScript	`if (stmt.step()) stmt.get()`
Python	`cur.fetchone()`
Java	`if (rs.next()) rs.getString(1)`

JDBC از 1 شمارش می‌کند؛ بقیه از 0.

خطای checked	
C#	ندارد
TypeScript	ندارد
Python	ندارد
Java	`SQLException` باید `catch` یا `throws` شود

در Java یا throws می‌نویسی یا به unchecked تبدیل می‌کنی (IllegalStateException).

ORM معمول	
C#	EF Core / Dapper
TypeScript	Prisma / TypeORM / Knex
Python	SQLAlchemy
Java	JPA (Hibernate) / JdbcTemplate

همه پارامتر می‌سازند؛ خطر injection وقتی برمی‌گردد که SQL خام را با رشته بسازید (FromSqlRaw، text()).

کد

مخزن پروژه‌ها روی SQLite

C# کد اجراشده و تست‌شده در /examples examples/backend/csharp/Portfolio/SqliteProjects.cs

```
public class SqliteProjects : IProjectRepository
{
    private readonly SqliteConnection conn;

    public SqliteProjects(SqliteConnection conn)
    {
        this.conn = conn;
        using var cmd = conn.CreateCommand();
        cmd.CommandText = "CREATE TABLE IF NOT EXISTS projects (key TEXT PRIMARY KEY, display_name TEXT NOT
NULL, adapter_base_url TEXT NOT NULL)";
        cmd.ExecuteNonQuery();
    }

    public async Task<ProjectRow?> FindAsync(string key)
    {
        await using var cmd = conn.CreateCommand();
        // $key is a parameter: the driver sends the value separately, never as SQL text.
        cmd.CommandText = "SELECT key, display_name, adapter_base_url FROM projects WHERE key = $key";
        cmd.Parameters.AddWithValue("$key", key);
        await using var reader = await cmd.ExecuteReaderAsync();
        return await reader.ReadAsync() ? new(reader.GetString(0), reader.GetString(1), reader.GetString(2)) :
null;
    }

    public async Task AddAsync(ProjectRow row)
    {
        await using var cmd = conn.CreateCommand();
        cmd.CommandText = "INSERT INTO projects (key, display_name, adapter_base_url) VALUES ($key, $name,
$url)";
        cmd.Parameters.AddWithValue("$key", row.Key);
        cmd.Parameters.AddWithValue("$name", row.DisplayName);
        cmd.Parameters.AddWithValue("$url", row.AdapterBaseUrl);
        await cmd.ExecuteNonQueryAsync();
    }
}
```

examples/backend/typescript/src/db.ts کد اجراشده و تست‌شده در /examples **TypeScript**

```typescript
export class SqliteProjects implements ProjectRepository {
  constructor(private readonly db: Database) {
    db.run(SCHEMA);
  }

  async find(key: string): Promise<ProjectRow | undefined> {
    // ? placeholders: the driver sends the value separately, never as SQL text.
    const stmt = this.db.prepare('SELECT key, display_name, adapter_base_url FROM projects WHERE key = ?');
    try {
      stmt.bind([key]);
      if (!stmt.step()) return undefined;
      const [k, displayName, adapterBaseUrl] = stmt.get() as string[];
      return { key: k, displayName, adapterBaseUrl };
    } finally {
      stmt.free(); // sql.js statements hold WebAssembly memory until freed
    }
  }

  async add(row: ProjectRow): Promise<void> {
    this.db.run('INSERT INTO projects (key, display_name, adapter_base_url) VALUES (?, ?, ?)', [
      row.key,
      row.displayName,
      row.adapterBaseUrl,
    ]);
  }
}
```

examples/backend/python/portfolio/db.py کد اجراشده و تست‌شده در /examples **Python**

```python
class SqliteProjects:
    def __init__(self, conn: sqlite3.Connection) -> None:
        self.conn = conn
        conn.execute(SCHEMA)

    def find(self, key: str) -> ProjectRow | None:
        # ? placeholders: the driver sends the value separately, never as SQL text.
        row = self.conn.execute(
            "SELECT key, display_name, adapter_base_url FROM projects WHERE key = ?", (key,)
        ).fetchone()
        return ProjectRow(key=row[0], display_name=row[1], adapter_base_url=row[2]) if row else None

    def add(self, row: ProjectRow) -> None:
        with self.conn:  # commits on success, rolls back on an exception
            self.conn.execute(
                "INSERT INTO projects (key, display_name, adapter_base_url) VALUES (?, ?, ?)",
                (row.key, row.display_name, row.adapter_base_url),
            )
```

Java کد اجراشده و تست‌شده در /examples

examples/backend/java/src/main/java/portfolio/SqliteProjects.java

```
public class SqliteProjects implements ProjectRepository {
    private final Connection conn;

    public SqliteProjects(Connection conn) throws SQLException {
        this.conn = conn;
        try (var st = conn.createStatement()) {
            st.execute("CREATE TABLE IF NOT EXISTS projects (key TEXT PRIMARY KEY, display_name TEXT NOT NULL,
adapter_base_url TEXT NOT NULL)");
        }
    }

    @Override
    public Optional<ProjectRow> find(String key) {
        // ? placeholders: the driver sends the value separately, never as SQL text.
        try (var ps = conn.prepareStatement("SELECT key, display_name, adapter_base_url FROM projects WHERE key
= ?")) {
            ps.setString(1, key);
            try (var rs = ps.executeQuery()) {
                return rs.next() ? Optional.of(new ProjectRow(rs.getString(1), rs.getString(2),
rs.getString(3))) : Optional.empty();
            }
        } catch (SQLException e) {
            throw new IllegalStateException(e); // checked -> unchecked, so the interface stays clean
        }
    }

    @Override
    public void add(ProjectRow row) {
        try (var ps = conn.prepareStatement("INSERT INTO projects (key, display_name, adapter_base_url) VALUES
(?, ?, ?)")) {
            ps.setString(1, row.key());
            ps.setString(2, row.displayName());
            ps.setString(3, row.adapterBaseUrl());
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new IllegalStateException(e);
        }
    }
}
```

اشتباه‌های رایج

همه **ساختن SQL با f-string، template literal یا +**

چرا: هر مقدار حاوی ' می‌تواند SQL را بشکند یا تغییر دهد (injection).

درست: همیشه پارامتر؛ حتی برای مقدارهایی که فکر می‌کنی امن‌اند.

Python **conn.execute(sql, (key))**

چرا: (key) یک رشته است، نه tuple؛ درایور هر حرف را یک پارامتر حساب می‌کند و خطای تعداد پارامتر می‌دهد.

درست: (key,) با ویرگول.

TypeScript **فراموش کردن stmt.free() در sql.js**

چرا: statement حافظه‌ی WebAssembly نگه می‌دارد که GC آزادش نمی‌کند؛ نشت حافظه.

درست: try/finally با free()؛ یا db.exec/db.run برای کوئری‌های یک‌باره.

C# **FromSqlRaw($"... {key}") در EF Core**

چرا: interpolation در FromSqlRaw رشته می‌سازد؛ injection.

درست: FromSql($"... {key}") یا FromSqlInterpolated که پارامتر می‌سازد.

Java **Statement.executeQuery با رشته‌ی ساخته‌شده**

چرا: Statement پارامتر ندارد.

درست: PreparedStatement با ?.

این کد کجای پروژه‌هاست

`ControlPanel/services/core-api/src/projects/projects.service.ts:55`

input('key', dto.key). :پارامتر در mssql

`ControlPanel/services/adapters/ledgerdash-adapter/src/app.ts:3`

خواندن فقط از viewها با لاگین ledger_reader

تمرین

▼ 1. در نسخه‌ی Python، find را عمداً با f-string بنویس و تست parameters_keep_injection را اجرا کن. چه اتفاقی می‌افتد؟

پاسخ: کوئری با خطای syntax می‌شکند یا (بسته به مقدار) نتیجه‌ی اشتباه می‌دهد؛ تست شکست می‌خورد. با پارامتر برمی‌گردد به حالت سبز.

▼ 2. یک کاربر SQL Server بساز که فقط روی یک view مجوز SELECT داشته باشد. چه دستورهایی لازم است؟

راهنما: CREATE LOGIN، CREATE USER، GRANT SELECT ON dbo.vName TO ...

پاسخ: CREATE LOGIN ledger_reader WITH PASSWORD = '...'; CREATE USER ledger_reader FOR LOGIN ledger_reader; GRANT SELECT ON dbo.vFollowupsDue TO ledger_reader; بدون نقش db_datareader، تا جدول‌ها دیده نشوند.

▼ 3. در Java چرا SQLException، SqliteProjects را به IllegalStateException تبدیل می‌کند؟

پاسخ: interface ProjectRepository متدهایش throws ندارد و نباید جزئیات JDBC را به اپ نشت دهد. تبدیل به unchecked باعث می‌شود خطا به ErrorHandler برسد و 500 بدهد.

جمله‌ی مصاحبه

Every value goes in as a parameter, every resource is closed, and the app's database login can only read what it needs.

درس 5 · backend

Validating input

اعتبارسنجی ورودی: شکل درست، و هیچ فیلد اضافه‌ای

validation | dto | mass assignment

بعد از این درس می‌توانی

- قواعد ورودی را روی یک DTO تعریف کنی (DataAnnotations، zod، Pydantic، Bean Validation).
- فیلدهای ناشناخته را رد کنی تا کسی نتواند فیلدی مثل isAdmin را جا بزند.
- در هر چهار زبان یک پاسخ 400 یکسان بدهی: { error, errors: { field: message } }.

سناریو از پروژه‌ی واقعی

core-api در Control Panel این خط را دارد: ValidationPipe({ whitelist: true, transform: true }). whitelist یعنی فیلدی که در DTO نیست دور ریخته می‌شود. ما یک قدم جلوتر می‌رویم و چنین فیلدی را خطا می‌دانیم، تا کلاینتی که فکر می‌کند isAdmin را تنظیم کرده بداند که نکرده. همین قاعده در هر چهار زبان ساخته و تست می‌شود.

سؤال دور ریختن فیلد اضافه کافی نیست؟

پاسخ از نظر امنیت کافی است. اما رد کردن آن اشتباه‌های کلاینت را زود نشان می‌دهد: اگر کسی displayname با n کوچک بفرستد، به‌جای اینکه بی‌صدا نادیده گرفته شود، 400 می‌گیرد.

سؤال FastAPI که خودش اعتبارسنجی می‌کند، چرا handler جدا؟

پاسخ FastAPI به‌طور پیش‌فرض 422 می‌دهد و شکل پاسخش متفاوت است. ما همه را 400 با یک شکل می‌کنیم تا کلاینت فرق زبان سرور را نبیند.

مفهوم‌ها

DTO جدا از مدل داده

CreateProject فقط چیزی است که کلاینت مجاز است بفرستد. ProjectRow چیزی است که ذخیره می‌شود. وقتی این دو یکی باشند، هر فیلد مدل (مثل نقش یا مالک) از بیرون قابل تنظیم می‌شود (mass assignment).

کجا قواعد نوشته می‌شوند

C# و Java روی property یا field با attribute/annotation. Pydantic روی type و zod .Field(...) یک شیء schema جدا که type را هم از آن درمی‌آورد (z.infer).

فیلد ناشناخته

Pydantic: extra="forbid". zod: .strict(). System.Text.Json: [JsonUnmappedMemberHandling(Disallow)]. Jackson: fail-on-unknown-properties=true (Spring Boot پیش‌فرضش را false گذاشته است).

اعتبارسنجی در Minimal API

Minimal API در C# خودش DataAnnotations را اجرا نمی‌کند (برخلاف کنترلرهای [ApiController]). کد ما Validator.TryValidateObject را صریحاً صدا می‌زند تا قاعده دیده شود.

مقایسه

تعریف DTO

C#	`class CreateProject { [Required] string Key }`
TypeScript	`z.object({ key: z.string() })`
Python	`class CreateProject(BaseModel)`
Java	`record CreateProject(@NotBlank String key)`

در zod schema اول است و type از آن ساخته می‌شود؛ در بقیه type اول است و قواعد روی آن.

regex

C#	`[RegularExpression("^...$")]`
TypeScript	`.regex(/^...$/)`
Python	`Field(pattern=r"^...$")`
Java	`@Pattern(regexp = "^...$")`

Pydantic v2 از موتور regex راست استفاده می‌کند؛ lookaround پشتیبانی نمی‌شود.

URL	
C#	`[Url]`
TypeScript	`z.string().url()`
Python	`AnyHttpUrl`
Java	`@URL(regexp = "^https?:.*")`

[Url] در C# ftp را هم قبول می‌کند؛ اگر فقط http(s) می‌خواهید قاعده‌ی جدا لازم است.

رد فیلد ناشناخته	
C#	`[JsonUnmappedMemberHandling(Disallow)]`
TypeScript	`.strict()`
Python	`model_config = ConfigDict(extra="forbid")`
Java	`fail-on-unknown-properties=true`

در Java سراسری است (یک property)، در بقیه روی هر DTO.

اجرای اعتبارسنجی	
C#	`Validator.TryValidateObject(...)`
TypeScript	`Schema.safeParse(req.body)`
Python	پارامتر type hint خودکار با
Java	`@Valid @RequestBody`

FastAPI و Spring خودکار اجرا می‌کنند؛ Express و Minimal API را خودتان صدا می‌زنید.

کد پیش‌فرض خطا	
C#	`400` (با `ValidationProblem`)
TypeScript	ندارد (خودتان)
Python	`422`
Java	`400`

FastAPI تنها کسی است که 422 می‌دهد؛ ما آن را 400 کردیم.

کد

DTO با قواعد

C# کد اجراشده و تست‌شده در /examples — examples/backend/csharp/Portfolio/Models.cs

```csharp
// Unknown fields are an error, not silently dropped (like Nest's whitelist + forbid).
[JsonUnmappedMemberHandling(JsonUnmappedMemberHandling.Disallow)]
public class CreateProject
{
    [Required, RegularExpression("^[a-z0-9-]{1,40}$")]
    public string Key { get; init; } = "";

    [Required, StringLength(100, MinimumLength = 1)]
    public string DisplayName { get; init; } = "";

    [Required, Url]
    public string AdapterBaseUrl { get; init; } = "";
}
```

TypeScript کد اجراشده و تست‌شده در /examples — examples/backend/typescript/src/models.ts

```typescript
export const CreateProject = z
  .object({
    key: z.string().regex(/^[a-z0-9-]{1,40}$/),
    displayName: z.string().min(1).max(100),
    adapterBaseUrl: z.string().url().refine((u) => /^https?:\/\//.test(u), 'must be http(s)'),
  })
  // Unknown fields are an error, not silently dropped (like Nest's whitelist + forbid).
  .strict();

export type CreateProject = z.infer<typeof CreateProject>;
```

Python کد اجراشده و تست‌شده در /examples

examples/backend/python/portfolio/models.py

```python
class CreateProject(BaseModel):
    # Unknown fields are an error, not silently dropped (like Nest's whitelist + forbid).
    model_config = ConfigDict(extra="forbid")

    key: str = Field(pattern=r"^[a-z0-9-]{1,40}$")
    display_name: str = Field(min_length=1, max_length=100)
    adapter_base_url: AnyHttpUrl
```

Java کد اجراشده و تست‌شده در /examples

examples/backend/java/src/main/java/portfolio/CreateProject.java

```java
// Unknown fields are rejected globally: spring.jackson.deserialization.fail-on-unknown-properties=true
public record CreateProject(
        @NotBlank @Pattern(regexp = "^[a-z0-9-]{1,40}$") String key,
        @NotBlank @Size(max = 100) String displayName,
        @NotBlank @URL(regexp = "^https?:.*") String adapterBaseUrl) {}
```

اشتباه‌های رایج

همه **گرفتن مستقیم entity دیتابیس به‌عنوان بدنه‌ی درخواست**

چرا: کلاینت هر فیلدی را می‌تواند تنظیم کند، از جمله نقش، مالک یا وضعیت (mass assignment).

درست: DTO جدا با فقط فیلدهای مجاز، و کپی صریح به مدل.

C# **انتظار اعتبارسنجی خودکار DataAnnotations در Minimal API**

چرا: Minimal API (تا قبل از AddValidation در .NET 10) آن را اجرا نمی‌کند؛ ورودی نامعتبر بی‌صدا رد می‌شود.

درست: Validator.TryValidateObject، یا endpoint filter، یا AddValidation در .NET 10.

Java **فراموش کردن @Valid کنار @RequestBody**

چرا: annotationهای record اجرا نمی‌شوند.

درست: @RequestBody @Valid.

TypeScript **استفاده از req.body بعد از safeParse به‌جای parsed.data**

چرا: req.body ممکن است فیلد اضافه یا نوع اشتباه داشته باشد؛ فقط parsed.data تضمین شده است.

درست: همیشه parsed.data.

این کد کجای پروژه‌هاست

ControlPanel/services/core-api/src/main.ts:17

ValidationPipe({ whitelist: true, transform: true })

`ControlPanel/services/core-api/src/projects/dto/project.dto.ts:4`

CreateProjectDto با class-validator

تمرین

▼ 1. یک قاعده اضافه کن: displayName نباید فقط فاصله باشد. در هر زبان کجا می‌نویسی؟

پاسخ: C#: [RegularExpression(@".*\S.*")]. zod: .trim().min(1). Pydantic: Field(min_length=1) با str_strip_whitespace یا یک field_validator. Java: @NotBlank که همین را چک می‌کند.

▼ 2. در NestJS whitelist: true فیلد اضافه را دور می‌ریزد. چه گزینه‌ای آن را خطا می‌کند؟

پاسخ: forbidNonWhitelisted: true. آن‌وقت ValidationPipe برای فیلد ناشناخته 400 می‌دهد.

▼ 3. تست unknown_field را در Java اجرا کن، بعد خط fail-on-unknown-properties را از application.properties بردار. نتیجه؟

پاسخ: تست شکست می‌خورد: Jackson فیلد isAdmin را نادیده می‌گیرد و پروژه با 201 ساخته می‌شود.

جمله‌ی مصاحبه

Input goes through a DTO that lists exactly what a client may send; unknown fields are rejected, and every language answers with the same 400 shape.

درس 6 · backend

Error handling without leaking

مدیریت خطا: جزئیات در لاگ، نه در پاسخ

error handling | logging | information leak

بعد از این درس می‌توانی

- یک error handler سراسری در هر چهار زبان بنویسی که خطای پیش‌بینی‌نشده را 500 با trace id برگرداند.
- پیام exception را هرگز به کاربر نشان ندهی، چون نام سرور، کاربر دیتابیس و SQL در آن است.
- خطاهای «عادی» (404، 409، 400) را از خطای واقعی جدا کنی.

سناریو از پروژه‌ی واقعی

تست ledgerdash-adapter دقیقاً همین را چک می‌کند: وقتی دیتابیس با «Login failed for user ledger_reader» می‌افتد، پاسخ فقط می‌گوید «LedgerDashboard database unreachable.». نام لاگین در پاسخ نیست. در این درس همین رفتار را برای هر خطای پیش‌بینی‌نشده‌ای در کل اپ می‌سازیم: کاربر یک trace id می‌گیرد، جزئیات با همان id در لاگ می‌روند.

سؤال پیام exception که به دیباگ کمک می‌کند. چرا مخفی‌اش کنیم؟

پاسخ چون مهاجم هم با آن دیباگ می‌کند: آدرس سرور، نام کاربر، ساختار جدول، نسخه‌ی کتابخانه. trace id به شما اجازه می‌دهد همان خطا را در لاگ پیدا کنید، بدون اینکه چیزی بیرون برود.

سؤال پس 404 و 409 هم باید مخفی شوند؟

پاسخ نه، آن‌ها خطای عمدی و قابل‌انتظارند و پیامشان را خودمان نوشته‌ایم. فقط خطای پیش‌بینی‌نشده پیام عمومی می‌گیرد.

مفهوم‌ها

دو نوع خطا

خطای عمدی (HttpError در TS، HTTPException در FastAPI، ResponseStatusException در Spring، Results.NotFound در C#) پیامی دارد که برای کاربر نوشته شده. هر چیز دیگر خطای پیش‌بینی‌نشده است و پیام عمومی می‌گیرد.

یک جا برای همه

FastAPI: middleware. Express: UseExceptionHandler# :C با چهار پارامتر که آخر ثبت می‌شود. Spring: @RestControllerAdvice. @app.exception_handler(Exception) با @ExceptionHandler. هر route لازم نیست try/catch داشته باشد.

Trace id

یک شناسه که هم در پاسخ و هم در لاگ می‌آید. کاربر آن را گزارش می‌دهد و شما در لاگ پیدا می‌کنید. ASP.NET خودش HttpContext.TraceIdentifier دارد؛ در بقیه یک UUID می‌سازیم.

Express: چرا چهار پارامتر؟

Express از تعداد پارامترهای تابع (function.length) تشخیص می‌دهد که middleware برای خطاست. اگر next را حذف کنید چون استفاده‌اش نمی‌کنید، Express آن را middleware معمولی حساب می‌کند.

مقایسه

handler سراسری	
C#	`app.UseExceptionHandler(...)`
TypeScript	`app.use((err, req, res, next) => ...)`
Python	`@app.exception_handler(Exception)`
Java	`@RestControllerAdvice + @ExceptionHandler`

در Express ترتیب مهم است: باید بعد از همه‌ی routeها ثبت شود.

خطای عمدی با کد	
C#	`Results.NotFound(...) (مقدار برگشتی)`
TypeScript	`throw new HttpError(404, ...)`
Python	`raise HTTPException(404, ...)`
Java	`throw new ResponseStatusException(NOT_FOUND, ...)`

C# در Minimal API معمولاً مقدار برمی‌گرداند، نه exception.

لاگ با جزئیات

C#	`logger.LogError(ex, "... {TraceId}", id)`
TypeScript	`console.error(`... ${id}`, err)`
Python	`logger.error("... %s", id, exc_info=exc)`
Java	`log.error("... {}", id, e)`

همه structured یا نیمه‌structured؛ در Java و C# exception آخرین آرگومان است.

body نامعتبر JSON

C#	`BadHttpRequestException` (با `ThrowOnBadRequest`)
TypeScript	`SyntaxError` از `express.json()`
Python	`RequestValidationError`
Java	`HttpMessageNotReadableException`

هر کدام نوع متفاوتی می‌اندازد؛ handler باید جدا آن را 400 کند.

کد

error handler سراسری

C# کد اجراشده و تست‌شده در /examples — examples/backend/csharp/Portfolio/PortfolioApp.cs

```
app.UseExceptionHandler(errorApp => errorApp.Run(async ctx =>
{
    var ex = ctx.Features.Get<IExceptionHandlerFeature>()?.Error;
    if (ex is BadHttpRequestException { InnerException: JsonException json })
    {
        var field = (json.Path ?? "$").TrimStart('$', '.');
        ctx.Response.StatusCode = 400;
        await ctx.Response.WriteAsJsonAsync(new { error = "Invalid request.", errors = new Dictionary<string,
string> { [field] = "Unknown or invalid field." } });
        return;
    }
    // The details go to the log, never to the caller (they can name servers, users, SQL).
    var traceId = ctx.TraceIdentifier;
    ctx.RequestServices.GetRequiredService<ILogger<WebApplication>>().LogError(ex, "Unhandled error {TraceId}",
traceId);
    ctx.Response.StatusCode = 500;
    await ctx.Response.WriteAsJsonAsync(new { error = "Something went wrong.", traceId });
}));
```

TypeScript کد اجراشده و تست‌شده در /examples examples/backend/typescript/src/app.ts

```ts
// Four parameters is how Express recognises an error handler. Registered last.
app.use((err: unknown, _req: Request, res: Response, _next: NextFunction) => {
  if (err instanceof HttpError) return res.status(err.status).json({ error: err.message });
  if (err instanceof SyntaxError) return res.status(400).json({ error: 'Body is not valid JSON.' });
  // The details go to the log, never to the caller (they can name servers, users, SQL).
  const traceId = randomUUID();
  console.error(`Unhandled error ${traceId}`, err);
  res.status(500).json({ error: 'Something went wrong.', traceId });
});
```

Python کد اجراشده و تست‌شده در /examples examples/backend/python/portfolio/app.py

```python
@app.exception_handler(RequestValidationError)
async def invalid(_req: Request, exc: RequestValidationError):
    # FastAPI answers 422 by default; we answer 400 like the other three languages.
    errors = {".".join(str(p) for p in e["loc"][1:]) or "body": e["msg"] for e in exc.errors()}
    return JSONResponse({"error": "Invalid request.", "errors": errors}, status_code=400)

@app.exception_handler(HTTPException)
async def http_error(_req: Request, exc: HTTPException):
    return JSONResponse({"error": exc.detail}, status_code=exc.status_code)

@app.exception_handler(Exception)
async def unexpected(_req: Request, exc: Exception):
    # The details go to the log, never to the caller (they can name servers, users, SQL).
    trace_id = uuid.uuid4().hex
    logger.error("Unhandled error %s", trace_id, exc_info=exc)
    return JSONResponse({"error": "Something went wrong.", "trace_id": trace_id}, status_code=500)
```

Java کد اجراشده و تست‌شده در /examples

examples/backend/java/src/main/java/portfolio/ErrorHandler.java

```
@RestControllerAdvice
public class ErrorHandler {
    private static final Logger log = LoggerFactory.getLogger(ErrorHandler.class);

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<Map<String, Object>> invalid(MethodArgumentNotValidException e) {
        var errors = new TreeMap<String, String>();
        e.getBindingResult().getFieldErrors().forEach(f -> errors.putIfAbsent(f.getField(),
f.getDefaultMessage()));
        return ResponseEntity.badRequest().body(Map.of("error", "Invalid request.", "errors", errors));
    }

    @ExceptionHandler(HttpMessageNotReadableException.class)
    ResponseEntity<Map<String, Object>> unreadable(HttpMessageNotReadableException e) {
        var field = e.getCause() instanceof UnrecognizedPropertyException u ? u.getPropertyName() : "body";
        return ResponseEntity.badRequest().body(Map.of("error", "Invalid request.", "errors", Map.of(field,
"Unknown or invalid field.")));
    }

    @ExceptionHandler(ResponseStatusException.class)
    ResponseEntity<Map<String, Object>> status(ResponseStatusException e) {
        return ResponseEntity.status(e.getStatusCode()).body(Map.of("error", e.getReason()));
    }

    @ExceptionHandler(Exception.class)
    ResponseEntity<Map<String, Object>> unexpected(Exception e) {
        // The details go to the log, never to the caller (they can name servers, users, SQL).
        var traceId = UUID.randomUUID().toString();
        log.error("Unhandled error {}", traceId, e);
        return ResponseEntity.internalServerError().body(Map.of("error", "Something went wrong.", "traceId",
traceId));
    }
}
```

اشتباه‌های رایج

همه **برگرداندن ex.Message یا err.message در پاسخ 500**

چرا: پیام exception اطلاعات داخلی دارد: آدرس سرور، نام کاربر، SQL، مسیر فایل.

درست: پیام عمومی + trace id؛ جزئیات فقط در لاگ.

C# **UseDeveloperExceptionPage یا ProblemDetails با جزئیات در Production**

چرا: stack trace کامل به کاربر می‌رسد.

درست: فقط در Development؛ در Production یک handler عمومی.

TypeScript **error middleware با سه پارامتر (بدون next)**

چرا: Express آن را middleware معمولی می‌بیند و هرگز با خطا صدا نمی‌زند.

درست: هر چهار پارامتر را بنویس، حتی _next.

Python **catch کردن Exception و return None در route**

چرا: FastAPI 200 با null برمی‌گرداند؛ خطا گم می‌شود.

درست: بگذار exception بالا برود تا handler سراسری آن را بگیرد.

این کد کجای پروژه‌هاست

`ControlPanel/services/adapters/ledgerdash-adapter/src/app.spec.ts:6`

تست: خطای دیتابیس به پاسخ نشت نمی‌کند

`ControlPanel/services/adapters/ledgerdash-adapter/src/app.ts:77`

«.insight: «Errors are reported, never leaked

تمرین

▼ 1. در هر زبان پیدا کن trace id در کجای لاگ می‌آید و چطور می‌توانی آن را از پاسخ به لاگ برسانی.

پاسخ: C#: TraceIdentifier هم در پاسخ و هم در پیام لاگ. TS و Python و Java: UUID ساخته‌شده در handler که در هر دو نوشته می‌شود. در لاگ جست‌وجوی همان رشته کافی است.

▼ 2. در error handler ،Express را قبل از routeها ثبت کن. تست 500 چه می‌شود؟

پاسخ: Express به‌جای handler ما از handler پیش‌فرض خودش استفاده می‌کند که HTML با stack trace (در development) برمی‌گرداند و تست شکل JSON را پیدا نمی‌کند.

▼ 3. چرا /status خطای دیتابیس را خودش می‌گیرد و به handler سراسری نمی‌سپارد؟

پاسخ: چون برای /status خرابی دیتابیس یک پاسخ معتبر است (healthy=false)، نه یک خطا. handler سراسری برای چیزی است که اصلاً انتظارش را نداشتیم.

جمله‌ی مصاحبه

Expected errors get a status code and a message I wrote; everything else becomes a 500 with a trace id, and the details stay in the log.

درس 7 · backend

Authentication with JWT and roles

احراز هویت با JWT: توکن کوتاه، توکن تمدید، و نقش‌ها

jwt | refresh token | rbac

بعد از این درس می‌توانی

- دو توکن بسازی: access برای ۱۵ دقیقه و refresh برای ۷ روز، با دو secret جدا.
- با TokenVersion امکان logout واقعی بدهی، بدون جدول توکن.
- نقش‌ها را یک بار تعریف کنی و روی route اعمال کنی: guard در Nest، filter در C#، interceptor در Spring، dependency در FastAPI.

سناریو از پروژه‌ی واقعی

core-api در Control Panel این طرح را دارد: access token پانزده دقیقه‌ای با typ: access، refresh token هفت‌روزه با secret جدا و typ: refresh و tv (TokenVersion). هر بار کاربر را دوباره از دیتابیس می‌خواند، پس تغییر نقش یا حذف کاربر بلافاصله اثر دارد. logout فقط TokenVersion را یکی زیاد می‌کند و همه‌ی refresh tokenهای قبلی باطل می‌شوند. اپ اگر secret کوتاه یا تکراری باشد اصلاً بالا نمی‌آید.

سؤال چرا دو secret؟ typ را که چک می‌کنیم.

پاسخ دفاع در عمق. اگر جایی چک typ فراموش شود، refresh token با secret دیگری امضا شده و اصلاً به‌عنوان access تأیید نمی‌شود. یک باگ کافی نیست؛ دو اشتباه لازم است.

سؤال access token را چطور قبل از ۱۵ دقیقه باطل کنیم؟

پاسخ در این طرح نمی‌شود؛ هزینه‌ی stateless بودن همین است. برای همین کوتاه است. اگر لازم است، باید در هر درخواست چیزی را در دیتابیس چک کنید و دیگر stateless نیست.

مفهوم‌ها

JWT چیست

سه بخش base64: header، payload (claims)، signature. هر کسی payload را می‌تواند بخواند؛ فقط صاحب secret می‌تواند امضا بسازد. پس داده‌ی محرمانه در JWT نگذار و نقش را فقط از توکن امضاشده بخوان، نه از بدنه‌ی درخواست.

الگوریتم را تو تعیین کن

حمله‌ی کلاسیک: توکنی با alg: none یا با الگوریتم دیگری که کتابخانه را گول بزند. Python و TS: algorithms=['HS256']. jjwt: verifyWith(SecretKey) C#: ValidAlgorithms فقط HMAC را قبول می‌کند.

TokenVersion

هر کاربر یک عدد دارد که در refresh token هم ذخیره می‌شود. logout یا تغییر رمز عدد را زیاد می‌کند. refresh با عدد قدیمی رد می‌شود. یک ستون، بدون جدول توکن.

نقش‌ها

Nest: @Roles('Admin') + RolesGuard که metadata را می‌خواند. Spring: annotation @RequireRole + HandlerInterceptor (همان ایده). C#: endpoint filter یا [Authorize(Roles = "Admin")] با JwtBearer. FastAPI: Depends(require_role("Admin")). 401 یعنی «نمی‌دانم کیستی»، 403 یعنی «می‌دانم، ولی اجازه نداری».

C#: MapInboundClaims

JwtSecurityTokenHandler به‌طور پیش‌فرض claim role را به URI طولانی ClaimTypes.Role تغییر نام می‌دهد. MapInboundClaims = false نام‌ها را همان‌طور نگه می‌دارد.

مقایسه

ساختن توکن	
C#	`handler.WriteToken(new JwtSecurityToken(...))`
TypeScript	`jwt.sign(payload, secret, { expiresIn: '15m' })`
Python	`jwt.encode(payload, secret, algorithm="HS256")`
Java	`Jwts.builder().claim(...).signWith(key).compact()`

در pyjwt خودتان exp را datetime می‌گذارید؛ jsonwebtoken رشته‌ای مثل '15m' می‌فهمد.

تأیید و قفل الگوریتم

C#	`ValidAlgorithms = [HmacSha256]`
TypeScript	`jwt.verify(t, s, { algorithms: ['HS256'] })`
Python	`jwt.decode(t, s, algorithms=["HS256"])`
Java	`Jwts.parser().verifyWith(key)`

pyjwt بدون algorithms خطا می‌دهد (خوب است)؛ jsonwebtoken بدون آن از header توکن پیروی می‌کند.

اعمال نقش روی route

C#	`.AddEndpointFilter(RequireRole("Admin"))`
TypeScript	`app.post(path, requireRole('Admin'), handler)`
Python	`dependencies=[Depends(require_role("Admin"))]`
Java	`@RequireRole("Admin") + HandlerInterceptor`

Express یک middleware در زنجیره است؛ Spring و Nest annotation/decorator را با reflection می‌خوانند.

معادل استاندارد فریم‌ورک

C#	`[Authorize(Roles = "Admin")] + AddJwtBearer`
TypeScript	`passport-jwt / Nest JwtAuthGuard`
Python	`fastapi.security.HTTPBearer`
Java	`Spring Security oauth2ResourceServer`

در پروژه‌ی واقعی از این‌ها استفاده کنید؛ مثال‌ها دستی‌اند تا مکانیزم دیده شود.

کد

TokenService

C# کد اجراشده و تست‌شده در /examples examples/backend/csharp/Portfolio/Tokens.cs

```
public class TokenService
{
    private const int MinSecretLength = 32;
    private static readonly TimeSpan AccessTtl = TimeSpan.FromMinutes(15);
    private static readonly TimeSpan RefreshTtl = TimeSpan.FromDays(7);

    private readonly SymmetricSecurityKey access, refresh;
    private readonly Func<int, User?> findUser;
    // MapInboundClaims = false keeps "role" as "role" instead of the long ClaimTypes.Role URI.
    private readonly JwtSecurityTokenHandler handler = new() { MapInboundClaims = false };

    // Two secrets: a refresh token can never verify as an access token, even before `typ` is read.
    public TokenService(string accessSecret, string refreshSecret, Func<int, User?> findUser)
    {
        if (Math.Min(accessSecret.Length, refreshSecret.Length) < MinSecretLength)
            throw new ArgumentException($"JWT secrets must be at least {MinSecretLength} characters.");
        if (accessSecret == refreshSecret)
            throw new ArgumentException("The access and refresh secrets must be different.");
        access = new(Encoding.UTF8.GetBytes(accessSecret));
        refresh = new(Encoding.UTF8.GetBytes(refreshSecret));
        this.findUser = findUser;
    }

    public TokenPair Issue(User user) => new(
        Write(access, AccessTtl, new("sub", user.Id.ToString()), new("role", user.Role), new("typ", "access")),
        Write(refresh, RefreshTtl, new("sub", user.Id.ToString()), new("typ", "refresh"),
            new("tv", user.TokenVersion.ToString(), ClaimValueTypes.Integer32)));

    public ClaimsPrincipal VerifyAccess(string token)
    {
        var principal = Verify(token, access, "Invalid or expired token.");
        if (principal.FindFirstValue("typ") != "access") throw new AuthException("Not an access token.");
        return principal;
    }

    public TokenPair Refresh(string token)
    {
        var principal = Verify(token, refresh, "Invalid or expired refresh token.");
        if (principal.FindFirstValue("typ") != "refresh") throw new AuthException("Not a refresh token.");
        // Re-read the user: a deleted user, a new role or a logout (TokenVersion bump) applies now.
        var user = findUser(int.Parse(principal.FindFirstValue("sub")!));
        if (user is null || user.TokenVersion.ToString() != principal.FindFirstValue("tv"))
            throw new AuthException("Refresh token has been revoked.");
        return Issue(user);
    }

    private string Write(SecurityKey key, TimeSpan ttl, params Claim[] claims) =>
        handler.WriteToken(new JwtSecurityToken(claims: claims, expires: DateTime.UtcNow.Add(ttl),
            signingCredentials: new SigningCredentials(key, SecurityAlgorithms.HmacSha256)));

    private ClaimsPrincipal Verify(string token, SecurityKey key, string error)
    {
        try
        {
            return handler.ValidateToken(token, new TokenValidationParameters
            {
                IssuerSigningKey = key,
                ValidAlgorithms = [SecurityAlgorithms.HmacSha256], // never let the token choose its algorithm
```

```
                ValidateIssuer = false,
                ValidateAudience = false,
                ClockSkew = TimeSpan.Zero,
            }, out _);
        }
        catch (Exception ex) when (ex is SecurityTokenException or ArgumentException)
        {
            throw new AuthException(error);
        }
    }
}
```

examples/backend/typescript/src/tokens.ts کد اجراشده و تست‌شده در /examples **TypeScript**

```
export class TokenService {
  // Two secrets: a refresh token can never verify as an access token, even before `typ` is read.
  constructor(
    private readonly accessSecret: string,
    private readonly refreshSecret: string,
    private readonly findUser: (id: number) => User | undefined,
  ) {
    if (Math.min(accessSecret.length, refreshSecret.length) < MIN_SECRET_LENGTH)
      throw new Error(`JWT secrets must be at least ${MIN_SECRET_LENGTH} characters.`);
    if (accessSecret === refreshSecret) throw new Error('The access and refresh secrets must be different.');
  }

  issue(user: User) {
    return {
      accessToken: jwt.sign({ sub: String(user.id), role: user.role, typ: 'access' }, this.accessSecret, {
        expiresIn: ACCESS_TTL,
        algorithm: 'HS256',
      }),
      refreshToken: jwt.sign({ sub: String(user.id), typ: 'refresh', tv: user.tokenVersion },
this.refreshSecret, {
        expiresIn: REFRESH_TTL,
        algorithm: 'HS256',
      }),
    };
  }

  verifyAccess(token: string): AccessClaims {
    let claims: jwt.JwtPayload;
    try {
      // algorithms is required: never let the token choose its own algorithm.
      claims = jwt.verify(token, this.accessSecret, { algorithms: ['HS256'] }) as jwt.JwtPayload;
    } catch {
      throw new AuthError('Invalid or expired token.');
    }
    if (claims.typ !== 'access') throw new AuthError('Not an access token.');
    return claims as unknown as AccessClaims;
  }

  refresh(token: string) {
    let claims: jwt.JwtPayload;
    try {
      claims = jwt.verify(token, this.refreshSecret, { algorithms: ['HS256'] }) as jwt.JwtPayload;
    } catch {
      throw new AuthError('Invalid or expired refresh token.');
    }
    if (claims.typ !== 'refresh') throw new AuthError('Not a refresh token.');
    // Re-read the user: a deleted user, a new role or a logout (tokenVersion bump) applies now.
    const user = this.findUser(Number(claims.sub));
    if (!user || user.tokenVersion !== claims.tv) throw new AuthError('Refresh token has been revoked.');
    return this.issue(user);
  }
}
```

examples/backend/python/portfolio/tokens.py کد اجراشده و تست‌شده در /examples **Python**

```python
class TokenService:
    # Two secrets: a refresh token can never verify as an access token, even before `typ` is read.
    def __init__(self, access_secret: str, refresh_secret: str, find_user: Callable[[int], User | None]):
        if min(len(access_secret), len(refresh_secret)) < MIN_SECRET_LENGTH:
            raise ValueError(f"JWT secrets must be at least {MIN_SECRET_LENGTH} characters.")
        if access_secret == refresh_secret:
            raise ValueError("The access and refresh secrets must be different.")
        self._access, self._refresh, self._find_user = access_secret, refresh_secret, find_user

    def issue(self, user: User) -> dict[str, str]:
        now = datetime.now(timezone.utc)
        access = {"sub": str(user.id), "role": user.role, "typ": "access", "exp": now + ACCESS_TTL}
        refresh = {"sub": str(user.id), "typ": "refresh", "tv": user.token_version, "exp": now + REFRESH_TTL}
        return {
            "access_token": jwt.encode(access, self._access, algorithm="HS256"),
            "refresh_token": jwt.encode(refresh, self._refresh, algorithm="HS256"),
        }

    def verify_access(self, token: str) -> dict:
        try:
            # algorithms= is required: never let the token choose its own algorithm.
            claims = jwt.decode(token, self._access, algorithms=["HS256"])
        except jwt.PyJWTError as e:
            raise AuthError("Invalid or expired token.") from e
        if claims.get("typ") != "access":
            raise AuthError("Not an access token.")
        return claims

    def refresh(self, token: str) -> dict[str, str]:
        try:
            claims = jwt.decode(token, self._refresh, algorithms=["HS256"])
        except jwt.PyJWTError as e:
            raise AuthError("Invalid or expired refresh token.") from e
        if claims.get("typ") != "refresh":
            raise AuthError("Not a refresh token.")
        # Re-read the user: a deleted user, a new role or a logout (token_version bump) applies now.
        user = self._find_user(int(claims["sub"]))
        if user is None or user.token_version != claims["tv"]:
            raise AuthError("Refresh token has been revoked.")
        return self.issue(user)
```

Java کد اجراشده و تست‌شده در examples/

examples/backend/java/src/main/java/portfolio/TokenService.java

```java
private static final int MIN_SECRET_LENGTH = 32;
private static final Duration ACCESS_TTL = Duration.ofMinutes(15);
private static final Duration REFRESH_TTL = Duration.ofDays(7);

private final SecretKey access;
private final SecretKey refresh;
private final IntFunction<User> findUser;

// Two secrets: a refresh token can never verify as an access token, even before `typ` is read.
public TokenService(String accessSecret, String refreshSecret, IntFunction<User> findUser) {
    if (Math.min(accessSecret.length(), refreshSecret.length()) < MIN_SECRET_LENGTH)
        throw new IllegalArgumentException("JWT secrets must be at least " + MIN_SECRET_LENGTH + "
characters.");
    if (accessSecret.equals(refreshSecret))
        throw new IllegalArgumentException("The access and refresh secrets must be different.");
    this.access = Keys.hmacShaKeyFor(accessSecret.getBytes(StandardCharsets.UTF_8));
    this.refresh = Keys.hmacShaKeyFor(refreshSecret.getBytes(StandardCharsets.UTF_8));
    this.findUser = findUser;
}

public TokenPair issue(User user) {
    var now = Instant.now();
    var accessToken = Jwts.builder().subject(String.valueOf(user.id))
            .claim("role", user.role).claim("typ", "access")
            .expiration(Date.from(now.plus(ACCESS_TTL))).signWith(access).compact();
    var refreshToken = Jwts.builder().subject(String.valueOf(user.id))
            .claim("typ", "refresh").claim("tv", user.tokenVersion)
            .expiration(Date.from(now.plus(REFRESH_TTL))).signWith(refresh).compact();
    return new TokenPair(accessToken, refreshToken);
}

public Claims verifyAccess(String token) {
    var claims = parse(token, access, "Invalid or expired token.");
    if (!"access".equals(claims.get("typ"))) throw new AuthException("Not an access token.");
    return claims;
}

public TokenPair refresh(String token) {
    var claims = parse(token, refresh, "Invalid or expired refresh token.");
    if (!"refresh".equals(claims.get("typ"))) throw new AuthException("Not a refresh token.");
    // Re-read the user: a deleted user, a new role or a logout (tokenVersion bump) applies now.
    var user = findUser.apply(Integer.parseInt(claims.getSubject()));
    if (user == null || !Objects.equals(user.tokenVersion, claims.get("tv", Integer.class)))
        throw new AuthException("Refresh token has been revoked.");
    return issue(user);
}

private static Claims parse(String token, SecretKey key, String error) {
    try {
        // verifyWith(SecretKey) only accepts HMAC-signed tokens: the token can't choose "none" or RSA.
        return Jwts.parser().verifyWith(key).build().parseSignedClaims(token).getPayload();
    } catch (JwtException | IllegalArgumentException e) {
        throw new AuthException(error);
```

```
    }
}
```

نقش روی route

C# کد اجراشده و تست‌شده در /examples — examples/backend/csharp/Portfolio/PortfolioApp.cs

```
private static Func<EndpointFilterInvocationContext, EndpointFilterDelegate, ValueTask<object?>>
RequireRole(params string[] roles) =>
    async (ctx, next) =>
    {
        var header = ctx.HttpContext.Request.Headers.Authorization.ToString();
        if (!header.StartsWith("Bearer ")) return Results.Json(new { error = "Sign in first." }, statusCode:
401);
        try
        {
            var tokens = ctx.HttpContext.RequestServices.GetRequiredService<TokenService>();
            var role = tokens.VerifyAccess(header["Bearer ".Length..]).FindFirst("role")?.Value;
            if (!roles.Contains(role)) return Results.Json(new { error = "Not allowed for your role." },
statusCode: 403);
        }
        catch (AuthException ex)
        {
            return Results.Json(new { error = ex.Message }, statusCode: 401);
        }
        return await next(ctx);
    };
```

TypeScript کد اجراشده و تست‌شده در /examples — examples/backend/typescript/src/app.ts

```
const requireRole =
  (...roles: User['role'][]) =>
  (req: Request, _res: Response, next: NextFunction) => {
    const header = req.header('authorization') ?? '';
    if (!header.startsWith('Bearer ')) return next(new HttpError(401, 'Sign in first.'));
    try {
      const claims = tokens.verifyAccess(header.slice('Bearer '.length));
      if (!roles.includes(claims.role)) return next(new HttpError(403, 'Not allowed for your role.'));
      next();
    } catch (err) {
      next(err instanceof AuthError ? new HttpError(401, err.message) : err);
    }
  };
```

Python کد اجراشده و تست‌شده در /examples

examples/backend/python/portfolio/app.py

```python
def require_role(*roles: str):
    def check(authorization: str | None = Header(default=None)) -> dict:
        if not authorization or not authorization.startswith("Bearer "):
            raise HTTPException(401, "Sign in first.")
        try:
            claims = tokens.verify_access(authorization.removeprefix("Bearer "))
        except AuthError as e:
            raise HTTPException(401, str(e)) from e
        if claims["role"] not in roles:
            raise HTTPException(403, "Not allowed for your role.")
        return claims

    return check
```

Java کد اجراشده و تست‌شده در /examples

examples/backend/java/src/main/java/portfolio/RequireRole.java

```java
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface RequireRole {
    String[] value();

    // Like Nest's RolesGuard: reads the annotation on the handler and checks the token's role.
    @Component
    class Interceptor implements HandlerInterceptor {
        private final TokenService tokens;

        public Interceptor(TokenService tokens) {
            this.tokens = tokens;
        }

        @Override
        public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) {
            if (!(handler instanceof HandlerMethod method)) return true;
            var required = method.getMethodAnnotation(RequireRole.class);
            if (required == null) return true;
            var header = req.getHeader("Authorization");
            if (header == null || !header.startsWith("Bearer "))
                throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Sign in first.");
            try {
                var role = tokens.verifyAccess(header.substring("Bearer ".length())).get("role", String.class);
                if (!Arrays.asList(required.value()).contains(role))
                    throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Not allowed for your role.");
                return true;
            } catch (TokenService.AuthException e) {
                throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, e.getMessage());
            }
        }
    }
}
```

اشتباه‌های رایج

TypeScript **jwt.verify بدون algorithms**

چرا: الگوریتم از header توکن خوانده می‌شود؛ پیکربندی اشتباه کلید می‌تواند به جعل منجر شود.

درست: { algorithms: ['HS256'] }.

همه **یک secret برای access و refresh**

چرا: refresh token هفت‌روزه به‌عنوان access token قبول می‌شود، اگر چک typ جایی جا بیفتد.

درست: دو secret متفاوت؛ اپ اگر برابر باشند بالا نیاید.

همه **secret پیش‌فرض در کد («changeme»)**

چرا: secret در ریپوی عمومی دیده می‌شود و هر کسی توکن می‌سازد.

درست: بدون مقدار پیش‌فرض؛ اپ بدون secret حداقل ۳۲ کاراکتری بالا نیاید (درس Docker، پیکربندی).

C# **خواندن نقش با FindFirst("role") وقتی MapInboundClaims روشن است**

چرا: claim به http://schemas.microsoft.com/.../role تغییر نام داده و null برمی‌گردد.

درست: MapInboundClaims = false یا ClaimTypes.Role.

همه **گرفتن نقش از بدنه یا query درخواست**

چرا: کلاینت هر چه بخواهد می‌فرستد.

درست: نقش فقط از توکن امضاشده یا دیتابیس.

این کد کجای پروژه‌هاست

`ControlPanel/services/core-api/src/auth/auth.service.ts:47`

refresh: دو typ، secret و TokenVersion

`ControlPanel/services/core-api/src/auth/jwt-config.ts:24`

secretها باید طولانی و متفاوت باشند

`ControlPanel/services/core-api/src/auth/roles.guard.ts:13`

RolesGuard: نقش از JWT، نه از درخواست

تمرین

▼ 1. یک refresh token بساز، TokenVersion کاربر را زیاد کن و refresh را صدا بزن. چه پیامی می‌گیری؟ این کدام عملیات واقعی را شبیه‌سازی می‌کند؟

پاسخ: Refresh token has been revoked. این همان logout یا تغییر رمز است.

▼ 2. access token را در jwt.io باز کن. چه چیزی را می‌بینی و چه چیزی را نمی‌توانی تغییر بدهی؟

پاسخ: header و payload (sub، role، typ، exp) را می‌بینی. اگر role را عوض کنی امضا دیگر معتبر نیست و سرور 401 می‌دهد.

▼ 3. در نسخه‌ی MapInboundClaims = false ،C# را بردار و تست Roles_are_checked را اجرا کن. چرا شکست می‌خورد؟

پاسخ: claim role به ClaimTypes.Role تغییر نام می‌دهد، FindFirst("role") null برمی‌گرداند و admin هم 403 می‌گیرد.

جمله‌ی مصاحبه

Short access tokens, refresh tokens with their own secret and a token version for real logout, and roles read only from the signed token.

درس 8 · backend

Testing the real app with fake dependencies

تست: اپ واقعی، دیتابیس جعلی

fakes | integration tests | testing

بعد از این درس می‌توانی

- تستی بنویسی که pipeline واقعی HTTP را اجرا کند (routing، اعتبارسنجی، error handler) و فقط دیتابیس را جعلی کند.
- ابزار هر زبان را بشناسی: JUnit + MockMvc، pytest + TestClient، node:test + fetch، xUnit + TestServer.
- تست‌هایی بنویسی که رفتار را چک می‌کنند، نه پیاده‌سازی را.

سناریو از پروژه‌ی واقعی

app.spec.ts در ledgerdash-adapter یک اپ Express واقعی را روی یک پورت آزاد بالا می‌آورد و با fetch صدایش می‌زند. فقط LedgerQueries جعلی است. پس تست همان چیزی را می‌بیند که core-api می‌بیند: JSON، کد وضعیت، و اینکه خطای دیتابیس نشت نمی‌کند. همین سه تست در چهار زبان پیاده شده‌اند و در هر push اجرا می‌شوند.

سؤال چرا mock کتابخانه‌ای (Moq، Mockito) استفاده نکردیم؟

پاسخ یک کلاس جعلی کوچک که port را پیاده می‌کند خواناتر است و با عوض شدن جزئیات داخلی نمی‌شکند. mock کتابخانه‌ای وقتی مفید است که بخواهی چک کنی چه چیزی صدا زده شد، که اینجا مهم نیست.

سؤال تست با سرور واقعی کند نیست؟

پاسخ TestServer در C# و MockMvc در Spring اصلاً پورت باز نمی‌کنند و در حافظه کار می‌کنند. Express روی پورت 0 در چند میلی‌ثانیه بالا می‌آید. کندی واقعی از دیتابیس واقعی است که جعلی شده.

مفهوم‌ها

هرم تست، ساده

تعداد زیادی تست سریع روی منطق خالص (backoff، تشخیص امضا)، تعدادی تست که اپ واقعی را با dependency جعلی اجرا می‌کنند (این درس)، و تعداد کمی تست روی زیرساخت واقعی (Mongo در CI).

Fake در برابر Mock

Fake یک پیاده‌سازی واقعی ولی ساده است (InMemoryProjects). Mock یک شیء است که انتظار دارد فلان متد با فلان آرگومان صدا زده شود. Fake تست را به رفتار وصل می‌کند، Mock به پیاده‌سازی.

اجرای اپ در تست

C#: UseTestServer + GetTestClient. TS: listen(0) (بدون پورت). Python: TestClient و fetch از Starlette. Java: MockMvc (بدون پورت) با @AutoConfigureMockMvc + @SpringBootTest.

تست‌های مستقل

هر تست اپ خودش را می‌سازد (یا وضعیت را در BeforeEach ریست می‌کند). هیچ تستی نباید به ترتیب اجرا یا داده‌ی تست قبلی وابسته باشد.

مقایسه

فریم‌ورک تست	
C#	`xUnit ([Fact])`
TypeScript	`node:test (describe/it)`
Python	`pytest (def test_...)`
Java	`JUnit 5 (@Test)`

pytest هیچ کلاس یا decorator لازم ندارد؛ هر تابع test_ یک تست است.

assert	
C#	`Assert.Equal(expected, actual)`
TypeScript	`assert.equal(actual, expected)`
Python	`assert actual == expected`
Java	`assertThat(actual).isEqualTo(expected)`

ترتیب expected/actual در xUnit و node:test برعکس است.

اجرای اپ

C#	`UseTestServer() + GetTestClient()`
TypeScript	`createApp(...).listen(0)`
Python	`TestClient(app)`
Java	`@AutoConfigureMockMvc + MockMvc`

فقط نسخه‌ی TS پورت واقعی باز می‌کند.

تست exception

C#	`Assert.Throws<T>(() => ...)`
TypeScript	`assert.throws(() => ..., Type)`
Python	`with pytest.raises(T):`
Java	`assertThatThrownBy(() -> ...).isInstanceOf(T.class)`

Python با context manager، بقیه با lambda.

skip شرطی

C#	`Skip / زودهنگام return`
TypeScript	`describe(name, { skip: ... })`
Python	`@pytest.mark.skipif(...)`
Java	`assumeTrue(...)`

تست Mongo بدون MONGO_URL اجرا نمی‌شود؛ در CI سرویس mongo دارد.

کد

سه تست /status

C# کد اجراشده و تست‌شده در /examples examples/backend/csharp/Portfolio.Tests/AppTests.cs

```
// The real ASP.NET pipeline in memory (TestServer); only the database is faked.
private static async Task<HttpClient> Client(IStatusQueries? queries = null, IProjectRepository? projects =
null)
{
    var app = PortfolioApp.Build(queries ?? new FakeQueries(), projects ?? new InMemoryProjects(), Tokens(),
        b => b.WebHost.UseTestServer());
    await app.StartAsync();
    return app.GetTestClient();
}

[Fact]
public async Task Status_reports_counts()
{
    var body = await (await Client()).GetFromJsonAsync<JsonElement>("/status");
    Assert.Equal("36 sent, 0 interviewing, 11 follow-up(s) due", body.GetProperty("summary").GetString());
}

[Fact]
public async Task Null_sums_count_as_zero()
{
    var body = await (await Client(new FakeQueries(new StatusCounts(null, null,
null)))).GetFromJsonAsync<JsonElement>("/status");
    Assert.Equal("0 sent, 0 interviewing, 0 follow-up(s) due", body.GetProperty("summary").GetString());
}

[Fact]
public async Task Unhealthy_without_leaking_the_database_error()
{
    var text = await (await Client(new FakeQueries(fail: true))).GetStringAsync("/status");
    Assert.Equal("""{"healthy":false,"error":"Database unreachable."}""", text);
}
```

TypeScript کد اجراشده و تست‌شده در /examples examples/backend/typescript/test/app.test.ts

```
// The real Express app on a free port; only the database is faked.
async function call(queries: StatusQueries, path: string, init?: RequestInit, projects = new
InMemoryProjects()) {
  const tokens = new TokenService(A, R, (id) => users.get(id));
  const server = createApp(queries, projects, tokens).listen(0);
  try {
    const { port } = server.address() as AddressInfo;
    const res = await fetch(`http://127.0.0.1:${port}${path}`, init);
    return { status: res.status, headers: res.headers, body: (await res.json()) as Record<string, any> };
  } finally {
    server.close();
  }
}

describe('/status', () => {
  it('reports counts', async () => {
    const { body } = await call(fakeQueries(), '/status');
    assert.equal(body.summary, '36 sent, 0 interviewing, 11 follow-up(s) due');
  });

  it('treats NULL sums as zero', async () => {
    const { body } = await call(fakeQueries({ sent: null, interviewing: null, followupsDue: null }),
'/status');
    assert.equal(body.summary, '0 sent, 0 interviewing, 0 follow-up(s) due');
  });

  it('is unhealthy without leaking the database error', async () => {
    const { body } = await call(fakeQueries(undefined, true), '/status');
    assert.deepEqual(body, { healthy: false, error: 'Database unreachable.' });
  });
});
```

Python کد اجراشده و تست‌شده در /examples examples/backend/python/tests/test_app.py

```
def test_status_reports_counts():
    client, _ = make()
    body = client.get("/status").json()
    assert body["healthy"] is True
    assert body["summary"] == "36 sent, 0 interviewing, 11 follow-up(s) due"


def test_null_sums_count_as_zero():
    client, _ = make(FakeQueries(StatusCounts()))
    assert client.get("/status").json()["summary"] == "0 sent, 0 interviewing, 0 follow-up(s) due"


def test_unhealthy_without_leaking_the_database_error():
    client, _ = make(FakeQueries(fail=True))
    body = client.get("/status").json()
    assert body == {"healthy": False, "error": "Database unreachable."}
    assert "ledger_reader" not in str(body)
```

Java کد اجراشده و تست‌شده در /examples examples/backend/java/src/test/java/portfolio/AppTest.java

```java
// The "test" profile skips the real configuration; these beans take its place.
@TestConfiguration
static class Fakes {
    @Bean FakeQueries queries() { return new FakeQueries(); }
    @Bean Broken projects() { return new Broken(); }
    @Bean TokenService tokens() { return new TokenService(A, R, USERS::get); }
}

@Autowired MockMvc mvc;
@Autowired FakeQueries queries;
@Autowired Broken projects;
@Autowired TokenService tokens;

@BeforeEach
void reset() {
    queries.counts = new StatusCounts(36, 0, 11);
    queries.fail = false;
    projects.on = false;
}

@Test
void statusReportsCounts() throws Exception {
    mvc.perform(get("/status")).andExpect(jsonPath("$.summary").value("36 sent, 0 interviewing, 11 follow-up(s)
due"));
}

@Test
void nullSumsCountAsZero() throws Exception {
    queries.counts = new StatusCounts(null, null, null);
    mvc.perform(get("/status")).andExpect(jsonPath("$.summary").value("0 sent, 0 interviewing, 0 follow-up(s)
due"));
}

@Test
void unhealthyWithoutLeakingTheDatabaseError() throws Exception {
    queries.fail = true;
    mvc.perform(get("/status"))
            .andExpect(jsonPath("$.healthy").value(false))
            .andExpect(jsonPath("$.error").value("Database unreachable."));
}
```

اشتباه‌های رایج

همه **تست‌هایی که فقط چک می‌کنند متدی صدا زده شد**

چرا: با هر refactor می‌شکنند، حتی وقتی رفتار عوض نشده.

درست: خروجی را چک کن: کد وضعیت، JSON، داده‌ی ذخیره‌شده.

Java **@SpringBootTest بدون جدا کردن beanهای واقعی**

چرا: context تست سعی می‌کند دیتابیس و secret واقعی را بسازد و بالا نمی‌آید.

درست: profile تست، یا @WebMvcTest با @MockitoBean.

TypeScript **بستن نکردن server در تست**

چرا: پروسه‌ی تست تمام نمی‌شود یا پورت‌ها پر می‌شوند.

درست: try/finally با server.close().

Python **TestClient(app) با raise_server_exceptions پیش‌فرض برای تست 500**

چرا: TestClient به‌جای پاسخ 500، خود exception را در تست می‌اندازد.

درست: raise_server_exceptions=False وقتی پاسخ 500 را تست می‌کنی.

این کد کجای پروژه‌هاست

`ControlPanel/services/adapters/ledgerdash-adapter/src/app.spec.ts:6`

اپ Express واقعی روی پورت آزاد؛ فقط دیتابیس جعلی

`ClinicHub/tests/ClinicHub.Tests/EncounterTests.cs:177`

تست هم‌زمانی با Task.WhenAll

تمرین

▼ 1. یک تست چهارم بنویس: وقتی metrics.sent برابر 1 است، summary باید «1 sent» باشد. در کدام زبان سریع‌تر نوشتی و چرا؟

پاسخ: هر جوابی درست است؛ نکته این است که در هر زبان فقط داده‌ی جعلی عوض می‌شود و اپ دست نمی‌خورد.

▼ 2. در Python raise_server_exceptions=False را از make() بردار. کدام تست شکست می‌خورد و چرا؟

پاسخ: test_unexpected_error_is_500_without_details؛ TestClient به‌جای پاسخ 500 خود RuntimeError را بالا می‌اندازد.

▼ 3. تمام تست‌های چهار زبان را اجرا کن. دستورها چیست؟

پاسخ: C#: dotnet test. TS: npm test. Python: pytest. Java: mvn test (یا در maven ... :Docker mvn test).

جمله‌ی مصاحبه

I test the real HTTP pipeline with fake ports, so tests check behaviour — status, JSON, no leaks — and survive refactoring.

درس 9 · backend

Atomic state changes in MongoDB

تغییر وضعیت اتمیک در MongoDB: فقط یک نفر برنده می‌شود

mongodb concurrency idempotency

بعد از این درس می‌توانی

- با یک findOneAndUpdate شرطی، یک تغییر وضعیت (pending → processing) را طوری بنویسی که دو درخواست هم‌زمان نتوانند هر دو انجامش بدهند.
- الگوی «بخوان، چک کن، بنویس» را بشناسی و بدانی چرا زیر بار شکست می‌خورد.
- همین مسئله را با تست واقعی روی Mongo ببینی: ده درخواست هم‌زمان، دقیقاً یک برنده.

سناریو از پروژه‌ی واقعی

در ReBiomed (بازار تجهیزات پزشکی) پرداخت Stripe از دو راه تأیید می‌شود: webhook و فراخوانی کلاینت. هر دو ممکن است هم‌زمان برسند. finalizeOrder فقط وقتی موجودی کالا را کم می‌کند که خودش سفارش را از pending به processing برده باشد، با یک findOneAndUpdate که شرط 'status: 'pending دارد. fulfillment.ts هم همین را برای ارسال و پرداخت به فروشنده انجام می‌دهد تا دوبار کلیک یا retry یک webhook دو بار پول جابه‌جا نکند.

سؤال اگر اول سفارش را بخوانم، چک کنم pending است و بعد آپدیت کنم، چه اشکالی دارد؟

پاسخ بین خواندن و نوشتن فاصله هست. دو درخواست هر دو pending را می‌خوانند و هر دو آپدیت می‌کنند. با فیلتر status در خود آپدیت، دیتابیس شرط و تغییر را یک قدم انجام می‌دهد.

سؤال تراکنش لازم نیست؟

پاسخ برای یک سند نه؛ هر عملیات روی یک سند در MongoDB اتمیک است. تراکنش برای وقتی است که چند سند باید با هم عوض شوند.

مفهوم‌ها

Compare-and-set

آپدیت شرطی همان compare-and-set است: «اگر وضعیت هنوز pending است، processing کن». هر کس آپدیت موفق بگیرد برنده است؛ بقیه null می‌گیرند و وضعیت فعلی را می‌خوانند.

Idempotency

عملیات idempotent است اگر دوبار اجرایش همان نتیجه‌ی یک بار را داشته باشد. finalizeOrder دوم چیزی را عوض نمی‌کند و فقط وضعیت فعلی را برمی‌گرداند. webhook retry و دوبار کلیک دیگر خطرناک نیستند.

همین در SQL

'UPDATE orders SET status='processing' WHERE id=@id AND status='pending' و بعد چک کردن تعداد سطرهای تغییرکرده (rowsAffected == 1). در EF Core: ExecuteUpdateAsync با Where، یا concurrency token.

returnDocument

پیش‌فرض findOneAndUpdate سند قبل از تغییر را برمی‌گرداند. returnDocument: after / ReturnDocument.AFTER سند بعد از تغییر را می‌دهد. برای تشخیص برنده هر دو کار می‌کند؛ مهم null بودن یا نبودن است.

مقایسه

آپدیت شرطی

C#	`FindOneAndUpdateAsync(o => o.Id == id && o.Status == "pending", ...)`
TypeScript	`findOneAndUpdate({ _id, status: 'pending' }, { $set: ... })`
Python	`find_one_and_update({"_id": id, "status": "pending"}, {"$set": ...})`
Java	`findOneAndUpdate(and(eq("_id", id), eq("status", "pending")), set(...))`

درایور C# فیلتر را از lambda می‌سازد؛ Java از Filters و Updates.

برگرداندن سند جدید

C#	`ReturnDocument = ReturnDocument.After`
TypeScript	`{ returnDocument: 'after' }`
Python	`return_document=ReturnDocument.AFTER`
Java	`new FindOneAndUpdateOptions().returnDocument(AFTER)`

در mongodb v6 برای Node، نتیجه مستقیم سند است (نه { value }).

نگاشت سند	
C#	`class OrderDoc با [BsonId]`
TypeScript	`interface OrderDoc + Collection<OrderDoc>`
Python	`dict`
Java	`org.bson.Document`

Python و Java اینجا بی‌type کار می‌کنند؛ Java می‌تواند POJO هم با CodecRegistry بگیرد.

اجرای هم‌زمان در تست	
C#	`Task.WhenAll(... Task.Run(...))`
TypeScript	`Promise.all(Array.from({ length: 10 }, ...))`
Python	`threading.Thread × 10`
Java	`CompletableFuture.supplyAsync × 10`

pymongo sync است، پس Python thread می‌سازد؛ بقیه async یا pool.

کد

pending → processing، فقط یک بار

C# کد اجراشده و تست‌شده در /examples examples/backend/csharp/Portfolio/Orders.cs

```
// Moves an order pending -> processing. Returns "won" for the one caller that did it,
// otherwise the status it already had (another request got there first).
public static async Task<string> FinalizeOrderAsync(IMongoCollection<OrderDoc> orders, string orderId)
{
    // One conditional update: the filter and the change happen as one atomic step on the server.
    var won = await orders.FindOneAndUpdateAsync(
        o => o.Id == orderId && o.Status == "pending",
        Builders<OrderDoc>.Update.Set(o => o.Status, "processing"),
        new FindOneAndUpdateOptions<OrderDoc> { ReturnDocument = ReturnDocument.After });
    if (won is not null) return "won";
    var current = await orders.Find(o => o.Id == orderId).FirstOrDefaultAsync()
        ?? throw new KeyNotFoundException($"Order {orderId} not found");
    return current.Status;
}
```

TypeScript کد اجراشده و تست‌شده در /examples examples/backend/typescript/src/orders.ts

```typescript
// Moves an order pending -> processing. Returns "won" for the one caller that did it,
// otherwise the status it already had (another request got there first).
export async function finalizeOrder(orders: Collection<OrderDoc>, orderId: string): Promise<string> {
  // One conditional update: the filter and the change happen as one atomic step on the server.
  const won = await orders.findOneAndUpdate(
    { _id: orderId, status: 'pending' },
    { $set: { status: 'processing' } },
    { returnDocument: 'after' },
  );
  if (won) return 'won';
  const current = await orders.findOne({ _id: orderId }, { projection: { status: 1 } });
  if (!current) throw new Error(`Order ${orderId} not found`);
  return current.status;
}
```

Python کد اجراشده و تست‌شده در /examples examples/backend/python/portfolio/orders.py

```python
def finalize_order(orders: Collection, order_id: str) -> str:
    """Moves an order pending -> processing. Returns "won" for the one caller that did it,
    otherwise the status it already had (another request got there first)."""
    # One conditional update: the filter and the change happen as one atomic step on the server.
    won = orders.find_one_and_update(
        {"_id": order_id, "status": "pending"},
        {"$set": {"status": "processing"}},
        return_document=ReturnDocument.AFTER,
    )
    if won is not None:
        return "won"
    current = orders.find_one({"_id": order_id}, {"status": 1})
    if current is None:
        raise LookupError(f"Order {order_id} not found")
    return current["status"]
```

Java کد اجراشده و تست‌شده در /examples examples/backend/java/src/main/java/portfolio/Orders.java

```java
// Moves an order pending -> processing. Returns "won" for the one caller that did it,
// otherwise the status it already had (another request got there first).
public static String finalizeOrder(MongoCollection<Document> orders, String orderId) {
    // One conditional update: the filter and the change happen as one atomic step on the server.
    var won = orders.findOneAndUpdate(
            and(eq("_id", orderId), eq("status", "pending")),
            set("status", "processing"),
            new FindOneAndUpdateOptions().returnDocument(ReturnDocument.AFTER));
    if (won != null) return "won";
    var current = orders.find(eq("_id", orderId)).projection(Projections.include("status")).first();
    if (current == null) throw new NoSuchElementException("Order " + orderId + " not found");
    return current.getString("status");
}
```

اشتباه‌های رایج

همه **خواندن سند، چک کردن وضعیت در کد، و بعد updateOne**

چرا: دو درخواست هم‌زمان هر دو چک را رد می‌کنند؛ موجودی دو بار کم یا پول دو بار پرداخت می‌شود.

درست: شرط را در فیلتر خود آپدیت بگذار.

TypeScript **خواندن result.value از findOneAndUpdate در driver v6**

چرا: از v6 نتیجه خود سند است و .value همیشه undefined است؛ هر درخواستی بازنده حساب می‌شود.

درست: خود نتیجه را چک کن (یا includeResultMetadata: true).

همه **فرض کردن اینکه webhook یک بار می‌رسد**

چرا: providerها در صورت timeout یا 5xx دوباره می‌فرستند.

درست: عملیات را idempotent کن (درس webhook هم event id را نگه می‌دارد).

این کد کجای پروژه‌هاست

`mern-animation-project/services/checkout.ts:152`

finalizeOrder در ReBiomed: فقط برنده‌ی processing → pending موجودی را کم می‌کند

`mern-animation-project/services/fulfillment.ts:13`

هر تغییر وضعیت یک آپدیت شرطی است

تمرین

▼ 1. finalize_order را عمداً به شکل «find_one، if status == pending، update_one» بنویس و تست هم‌زمانی را چند بار اجرا کن. چه می‌بینی؟

پاسخ: گاهی بیش از یک «won» دیده می‌شود (بسته به زمان‌بندی). این همان race condition است که آپدیت شرطی حذف می‌کند.

▼ 2. معادل SQL این تابع را برای SQL Server بنویس و بگو برنده را چطور تشخیص می‌دهی.

پاسخ: 'UPDATE dbo.Orders SET Status = 'processing' WHERE Id = @id AND Status = 'pending'; اگر @@ROWCOUNT (یا rowsAffected) برابر 1 بود برنده‌ای، اگر 0 بود کس دیگری قبلاً انجام داده.

▼ 3. چرا تست Mongo در CI اجرا می‌شود ولی روی لپ‌تاپ بدون MONGO_URL skip می‌شود؟

پاسخ: چون به سرور Mongo واقعی نیاز دارد. CI یک سرویس mongo بالا می‌آورد؛ روی لپ‌تاپ با docker run -p 27017:27017 mongo:7 و MONGO_URL اجرا می‌شود.

جمله‌ی مصاحبه

A state change is one conditional update — 'only if still pending' — so concurrent requests and retried webhooks can't do the work twice.

درس 10 · backend

Receiving webhooks safely

دریافت امن webhook: بدنه‌ی خام، امضا، تکرار

webhooks | hmac | raw body | replay

بعد از این درس می‌توانی

- امضای HMAC یک webhook را روی بدنه‌ی خام بررسی کنی، با مقایسه‌ی زمان‌ثابت.
- timestampهای قدیمی را رد کنی تا پیام ضبط‌شده دوباره قابل استفاده نباشد.
- یک event را فقط یک بار پردازش کنی، حتی وقتی provider آن را دوباره می‌فرستد.

سناریو از پروژه‌ی واقعی

در ReBiomed، مسیر api/payment/webhook/ در app.ts قبل از express.json() ثبت شده، با این توضیح: «Stripe signs the raw request body». اگر JSON اول parse شود و بعد دوباره رشته شود، فاصله‌ها و ترتیب کلیدها عوض می‌شوند و امضا دیگر نمی‌خواند. verify هم دو secret را امتحان می‌کند، چون دو endpoint در Stripe (پرداخت و Connect) هر کدام secret خودشان را دارند. مثال‌های ما همان طرح Stripe را (t=…,v1=HMAC) بدون SDK پیاده می‌کنند تا مکانیزم دیده شود.

سؤال چرا مقایسه‌ی رشته‌ی عادی (==) برای امضا کافی نیست؟

پاسخ == در اولین کاراکتر متفاوت متوقف می‌شود. با اندازه‌گیری زمان پاسخ، مهاجم می‌تواند امضا را کاراکتر به کاراکتر حدس بزند. مقایسه‌ی زمان‌ثابت همیشه همه‌ی بایت‌ها را می‌خواند.

سؤال امضا که درست است، timestamp دیگر چه کاری می‌کند؟

پاسخ یک پیام درست و امضاشده را می‌شود ضبط کرد و یک ماه بعد دوباره فرستاد. timestamp داخل چیزی است که امضا شده، پس قابل تغییر نیست، و بیش از ۵ دقیقه اختلاف یعنی رد.

مفهوم‌ها

بدنه‌ی خام

امضا روی بایت‌هایی است که provider فرستاده. express.raw :Express قبل از express.json. ASP.NET: Request.Body را مستقیم بخوان. Spring: @RequestBody byte[]. FastAPI: await request.body().

HMAC

HMAC-SHA256(secret, "<timestamp>.<body>"). هر کسی که secret را دارد می‌تواند امضا بسازد و هیچ کس دیگری نمی‌تواند. secret را هرگز در کد یا لاگ نگذار.

مقایسه‌ی زمان‌ثابت

Python: hmac.compare_digest. Node: crypto.timingSafeEqual (که با طول متفاوت throw می‌کند، پس اول طول را چک کن). C#: CryptographicOperations.FixedTimeEquals. Java: MessageDigest.isEqual.

یک بار پردازش

provider هر بار که پاسخ 2xx سریع نگیرد دوباره می‌فرستد. id رویداد را نگه دار و تکراری را با 200 جواب بده (نه خطا، وگرنه باز می‌فرستد). در مثال یک set در حافظه است؛ در تولید یک جدول با کلید یکتا یا آپدیت شرطی درس قبل.

مقایسه

خواندن بدنه‌ی خام

C#	`await request.Body.CopyToAsync(buffer)`
TypeScript	`express.raw({ type: 'application/json' })`
Python	`await request.body()`
Java	`@RequestBody byte[] body`

در Express اگر express.json() زودتر ثبت شده باشد، بدنه‌ی خام دیگر در دسترس نیست.

HMAC-SHA256

C#	`HMACSHA256.HashData(key, data)`
TypeScript	`createHmac('sha256', secret).update(...).digest('hex')`
Python	`hmac.new(key, data, hashlib.sha256).hexdigest()`
Java	`Mac.getInstance("HmacSHA256")`

C# و Python یک فراخوانی؛ Java سه قدم (getInstance، init، doFinal).

مقایسه‌ی زمان‌ثابت

C#	`CryptographicOperations.FixedTimeEquals(a, b)`
TypeScript	`timingSafeEqual(a, b)`
Python	`hmac.compare_digest(a, b)`
Java	`MessageDigest.isEqual(a, b)`

timingSafeEqual در Node با طول نابرابر exception می‌دهد.

SDK رسمی Stripe

C#	`EventUtility.ConstructEvent(json, sig, secret)`
TypeScript	`stripe.webhooks.constructEvent(body, sig, secret)`
Python	`stripe.Webhook.construct_event(body, sig, secret)`
Java	`Webhook.constructEvent(payload, sig, secret)`

در پروژه‌ی واقعی SDK را صدا بزن؛ همان کار مثال‌ها را می‌کند.

کد

بررسی امضا

C# کد اجراشده و تست‌شده در /examples examples/backend/csharp/Portfolio/Webhook.cs

```
public static string Sign(string secret, long timestamp, byte[] body)
{
    // Stripe-style: HMAC-SHA256 over "<timestamp>.<raw body>".
    byte[] payload = [.. Encoding.UTF8.GetBytes($"{timestamp}."), .. body];
    return Convert.ToHexStringLower(HMACSHA256.HashData(Encoding.UTF8.GetBytes(secret), payload));
}

public static JsonElement Verify(byte[] body, string header, IEnumerable<string> secrets, long now)
{
    var parts = header.Split(',').Select(p => p.Split('=', 2)).Where(p => p.Length == 2).ToDictionary(p =>
p[0], p => p[1]);
    if (!long.TryParse(parts.GetValueOrDefault("t"), out var timestamp))
        throw new SignatureException("No timestamp in the signature header.");
    if (Math.Abs(now - timestamp) > ToleranceSeconds)
        throw new SignatureException("Timestamp outside the tolerance (replayed?).");
    var given = Encoding.UTF8.GetBytes(parts.GetValueOrDefault("v1") ?? "");
    foreach (var secret in secrets) // two endpoints, two secrets: accept either
    {
        // FixedTimeEquals takes the same time whatever matches, so timing leaks nothing.
        if (CryptographicOperations.FixedTimeEquals(Encoding.UTF8.GetBytes(Sign(secret, timestamp, body)),
given))
            return JsonDocument.Parse(body).RootElement.Clone();
    }
    throw new SignatureException("No matching signature.");
}
```

TypeScript کد اجراشده و تست‌شده در /examples examples/backend/typescript/src/webhook.ts

```
export function sign(secret: string, timestamp: number, body: Buffer): string {
  // Stripe-style: HMAC-SHA256 over "<timestamp>.<raw body>".
  return createHmac('sha256', secret).update(`${timestamp}.`).update(body).digest('hex');
}

export function verify(body: Buffer, header: string, secrets: string[], now: number): { id: string } {
  const parts = Object.fromEntries(header.split(',').map((p) => p.split('=', 2) as [string, string]));
  const timestamp = Number(parts.t);
  if (!Number.isInteger(timestamp)) throw new SignatureError('No timestamp in the signature header.');
  if (Math.abs(now - timestamp) > TOLERANCE_SECONDS) throw new SignatureError('Timestamp outside the tolerance
(replayed?).');
  const given = Buffer.from(parts.v1 ?? '', 'hex');
  for (const secret of secrets) {
    const expected = Buffer.from(sign(secret, timestamp, body), 'hex');
    // timingSafeEqual takes the same time whatever matches; it throws on different lengths, so check first.
    if (given.length === expected.length && timingSafeEqual(given, expected)) return
JSON.parse(body.toString('utf8'));
  }
  throw new SignatureError('No matching signature.');
}
```

examples/backend/python/portfolio/webhook.py کد اجراشده و تست‌شده در /examples **Python**

```python
def sign(secret: str, timestamp: int, body: bytes) -> str:
    # Stripe-style: HMAC-SHA256 over "<timestamp>.<raw body>".
    return hmac.new(secret.encode(), f"{timestamp}.".encode() + body, hashlib.sha256).hexdigest()


def verify(body: bytes, header: str, secrets: list[str], now: float) -> dict:
    parts = dict(p.split("=", 1) for p in header.split(",") if "=" in p)
    try:
        timestamp = int(parts.get("t", ""))
    except ValueError as e:
        raise SignatureError("No timestamp in the signature header.") from e
    if abs(now - timestamp) > TOLERANCE_SECONDS:
        raise SignatureError("Timestamp outside the tolerance (replayed?).")
    for secret in secrets:  # two endpoints, two secrets: accept either
        # compare_digest takes the same time whatever matches, so timing leaks nothing.
        if hmac.compare_digest(sign(secret, timestamp, body), parts.get("v1", "")):
            return json.loads(body)
    raise SignatureError("No matching signature.")
```

Java کد اجراشده و تست‌شده در /examples examples/backend/java/src/main/java/portfolio/Webhook.java

```java
public static String sign(String secret, long timestamp, byte[] body) {
    // Stripe-style: HMAC-SHA256 over "<timestamp>.<raw body>".
    try {
        var mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(secret.getBytes(StandardCharsets.UTF_8), "HmacSHA256"));
        mac.update((timestamp + ".").getBytes(StandardCharsets.UTF_8));
        return HexFormat.of().formatHex(mac.doFinal(body));
    } catch (GeneralSecurityException e) {
        throw new IllegalStateException(e);
    }
}

public static JsonNode verify(byte[] body, String header, List<String> secrets, long now) throws
SignatureException {
    Map<String, String> parts = new HashMap<>();
    for (var p : header.split(",")) {
        var kv = p.split("=", 2);
        if (kv.length == 2) parts.put(kv[0], kv[1]);
    }
    long timestamp;
    try {
        timestamp = Long.parseLong(parts.getOrDefault("t", ""));
    } catch (NumberFormatException e) {
        throw new SignatureException("No timestamp in the signature header.");
    }
    if (Math.abs(now - timestamp) > TOLERANCE_SECONDS)
        throw new SignatureException("Timestamp outside the tolerance (replayed?).");
    var given = parts.getOrDefault("v1", "").getBytes(StandardCharsets.UTF_8);
    for (var secret : secrets) { // two endpoints, two secrets: accept either
        // MessageDigest.isEqual takes the same time whatever matches, so timing leaks nothing.
        if (MessageDigest.isEqual(sign(secret, timestamp, body).getBytes(StandardCharsets.UTF_8), given)) {
            try {
                return MAPPER.readTree(body);
            } catch (IOException e) {
                throw new SignatureException("Body is not JSON.");
            }
        }
    }
    throw new SignatureException("No matching signature.");
}
```

endpoint: بدنه‌ی خام و یک بار پردازش

C# کد اجراشده و تست‌شده در /examples examples/backend/csharp/Portfolio/Webhook.cs

```
public static void MapPaymentsWebhook(this WebApplication app, string[] secrets, Func<JsonElement, Task>
handle,
    ISet<string> seen, Func<long>? clock = null)
{
    clock ??= () => DateTimeOffset.UtcNow.ToUnixTimeSeconds();
    app.MapPost("/webhooks/payments", async (HttpRequest request) =>
    {
        // The raw bytes: binding to a typed object first would change what was signed.
        using var buffer = new MemoryStream();
        await request.Body.CopyToAsync(buffer);
        JsonElement evt;
        try
        {
            evt = Verify(buffer.ToArray(), request.Headers["Stripe-Signature"].ToString(), secrets, clock());
        }
        catch (SignatureException ex)
        {
            return Results.BadRequest(new { error = $"Signature verification failed: {ex.Message}" });
        }
        // Providers retry: the same event can arrive twice. Handle it once.
        var id = evt.GetProperty("id").GetString()!;
        if (seen.Contains(id)) return Results.Ok(new { received = true, duplicate = true });
        await handle(evt);
        seen.Add(id);
        return Results.Ok(new { received = true });
    });
}
```

TypeScript از کد واقعی پروژه (کوتاه‌شده)

```ts
// ReBiomed app.ts: Stripe signs the raw request body, so the webhook must be mounted before express.json().
app.use('/api/payment/webhook', webhookRoutes);

// routes/webhook.ts
router.post('/', express.raw({ type: 'application/json' }), async (req: Request, res: Response) => {
  if (!keys.stripeWebhookSecret && !keys.stripeConnectWebhookSecret) {
    return res.status(503).json({ msg: 'Webhook secret not configured' });
  }

  let event: Stripe.Event;
  try {
    event = verify(req.body, req.header('stripe-signature') ?? '');
  } catch (err) {
    return res.status(400).json({ msg: `Webhook signature verification failed: ${(err as Error).message}` });
  }

  try {
    if (event.type === 'payment_intent.succeeded') {
      const pi = event.data.object;
      if (pi.metadata.orderId) await finalizeOrder(pi.metadata.orderId, pi, refund);
    } else if (event.type === 'payment_intent.payment_failed') {
      await markOrderFailed(event.data.object.id);
    }
    res.json({ received: true });
  } catch (err) {
    console.error((err as Error).message);
    res.status(500).json({ msg: 'Webhook handling failed' });
  }
});
```

Python کد اجراشده و تست‌شده در /examples examples/backend/python/portfolio/webhook.py

```python
def webhook_router(
    secrets: list[str], handle: Callable[[dict], None], seen: set[str], clock: Callable[[], float] = time.time
) -> APIRouter:
    router = APIRouter()

    @router.post("/webhooks/payments")
    async def receive(request: Request, stripe_signature: str = Header(default="")):
        body = await request.body()  # the raw bytes: parsing first would change what was signed
        try:
            event = verify(body, stripe_signature, secrets, clock())
        except SignatureError as e:
            raise HTTPException(400, f"Signature verification failed: {e}") from e
        # Providers retry: the same event can arrive twice. Handle it once.
        if event["id"] in seen:
            return {"received": True, "duplicate": True}
        handle(event)
        seen.add(event["id"])
        return {"received": True}

    return router
```

Java کد اجراشده و تست‌شده در /examples examples/backend/java/src/main/java/portfolio/Webhook.java

```
public record Settings(List<String> secrets, Consumer<JsonNode> handle, Set<String> seen, LongSupplier clock)
{}

@RestController
@Profile("!test") // tests build it themselves with their own Settings
public static class Controller {
    private final Settings settings;

    public Controller(Settings settings) {
        this.settings = settings;
    }

    // byte[] body: the raw bytes, because binding to a typed object first would change what was signed.
    @PostMapping("/webhooks/payments")
    public ResponseEntity<Map<String, Object>> receive(@RequestBody byte[] body,
            @RequestHeader(value = "Stripe-Signature", defaultValue = "") String signature) {
        JsonNode event;
        try {
            event = verify(body, signature, settings.secrets(), settings.clock().getAsLong());
        } catch (SignatureException e) {
            return ResponseEntity.badRequest().body(Map.of("error", "Signature verification failed: " +
e.getMessage()));
        }
        // Providers retry: the same event can arrive twice. Handle it once.
        var id = event.get("id").asText();
        if (settings.seen().contains(id)) return ResponseEntity.ok(Map.of("received", true, "duplicate",
true));
        settings.handle().accept(event);
        settings.seen().add(id);
        return ResponseEntity.ok(Map.of("received", true));
    }
}
```

اشتباه‌های رایج

TypeScript **ثبت express.json() قبل از مسیر webhook**

چرا: بدنه parse شده و دیگر بایت‌های اصلی نیست؛ امضا هرگز نمی‌خواند.

درست: مسیر webhook را با express.raw قبل از express.json ثبت کن.

همه **مقایسه‌ی امضا با == یا equals**

چرا: زمان پاسخ اطلاعات می‌دهد (timing attack).

درست: تابع مقایسه‌ی زمان‌ثابت زبان.

همه **برگرداندن 500 برای event تکراری**

چرا: provider دوباره و دوباره می‌فرستد.

درست: 200 با duplicate: true.

همه **کار سنگین قبل از پاسخ دادن**

چرا: provider بعد از چند ثانیه timeout می‌کند و دوباره می‌فرستد.

درست: ذخیره کن، سریع 200 بده، پردازش را در پس‌زمینه انجام بده (مثل outbox).

این کد کجای پروژه‌هاست

`mern-animation-project/app.ts:31`

ReBiomed: webhook قبل از express.json() ثبت می‌شود

`mern-animation-project/routes/webhook.ts:18`

verify با دو secret (پرداخت و Connect)

`mern-animation-project/routes/webhook.ts:35`

express.raw و بررسی امضا

تمرین

▼ 1. در تست webhook، بدنه را بعد از امضا فقط با اضافه کردن یک فاصله تغییر بده. نتیجه؟

پاسخ: 400؛ HMAC روی بایت‌هاست و یک فاصله هم امضا را عوض می‌کند. به همین دلیل بدنه باید خام بماند.

▼ 2. tolerance را از 300 به 0 تغییر بده و تست valid را اجرا کن. چرا هنوز پاس می‌شود؟

پاسخ: چون clock در تست ثابت است (1_000_000) و timestamp هم همان است. در تولید با tolerance صفر، تأخیر شبکه هر پیامی را رد می‌کند.

▼ 3. set حافظه‌ای seen را با یک جدول SQL جایگزین کن. چه قیدی روی جدول لازم است و چرا؟

پاسخ: PRIMARY KEY یا UNIQUE روی event_id. INSERT دوم با خطای کلید تکراری شکست می‌خورد و همان نشان «تکراری» است، حتی بین چند نمونه از سرور.

جمله‌ی مصاحبه

I verify the HMAC over the raw body with a constant-time compare, reject old timestamps, and handle each event id once.

درس 11 · backend

The outbox pattern

الگوی Outbox: پیام با داده ذخیره می‌شود، بعداً فرستاده می‌شود

outbox retry backoff transactions

بعد از این درس می‌توانی

- سفارش و پیامش را در یک تراکنش ذخیره کنی تا هیچ‌وقت یکی بدون دیگری نباشد.
- یک dispatcher بنویسی که پیام‌های آماده را بفرستد و در خطا با فاصله‌ی رو به رشد دوباره امتحان کند.
- بدانی کی بعد از چند تلاش باید دست کشید و وضعیت را شکست‌خورده نشان داد.

سناریو از پروژه‌ی واقعی

ClinicHub وقتی سفارش آزمایش یا رادیولوژی ثبت می‌کند، پیام FHIR آن را در جدول Outbox همان دیتابیس کلینیک می‌گذارد و هر دو با هم commit می‌شوند. OutboxWorker هر چند ثانیه پیام‌های آماده را به LabFlow یا ClinicReport می‌فرستد. اگر طرف مقابل خاموش باشد، پیام می‌ماند و بعد از ۱۰، ۲۰، ۴۰ ثانیه... تا ۳۰ دقیقه دوباره امتحان می‌شود. بعد از ۱۲ تلاش سفارش Failed می‌شود. این درس همان طرح را با SQLite در چهار زبان نشان می‌دهد.

سؤال چرا پیام را همان لحظه‌ی ثبت سفارش با HTTP نفرستیم؟

پاسخ دو حالت بد پیش می‌آید: سفارش ذخیره شد ولی HTTP شکست خورد (پیام گم شد)، یا HTTP رفت ولی ذخیره شکست خورد (طرف مقابل سفارشی دارد که ما نداریم). با outbox هر دو در یک تراکنش‌اند.

سؤال طرف مقابل ممکن است پیام را دو بار بگیرد؟

پاسخ بله؛ اگر ارسال موفق باشد ولی ثبت sent قبل از crash انجام نشود. outbox «حداقل یک بار» تحویل می‌دهد، پس گیرنده باید idempotent باشد (درس ۹ و ۱۰).

مفهوم‌ها

یک تراکنش، دو سطر

INSERT سفارش و INSERT پیام در یک تراکنش. اگر هر کدام شکست بخورد، هیچ کدام نمی‌ماند. تست این را با یک id تکراری نشان می‌دهد: تعداد پیام‌ها ۱ می‌ماند.

Dispatcher

پیام‌هایی را می‌خواند که sent ندارند، تلاششان کمتر از سقف است، و زمان تلاش بعدی‌شان رسیده. هر پیام را می‌فرستد؛ موفق → sent = now؛ ناموفق → attempts++ و next_attempt = now + backoff.

Exponential backoff با سقف

10 × 2^(n-1) ثانیه: 10، 20، 40، 80… تا حداکثر ۳۰ دقیقه. طرف مقابلِ در حال ریکاوری زیر بار retryهای پشت سر هم نمی‌ماند.

زمان را تزریق کن

dispatch پارامتر now می‌گیرد، به‌جای اینکه خودش ساعت را بخواند. تست می‌تواند بگوید «الان ثانیه‌ی 10 است» بدون اینکه صبر کند. ClinicHub همین کار را با TimeProvider انجام می‌دهد.

چه کسی dispatch را صدا می‌زند

همان کارگر پس‌زمینه‌ی درس ۱: @Scheduled ،BackgroundService، یا حلقه‌ی async. یک نمونه کافی است؛ با چند نمونه باید قفل یا SELECT ... FOR UPDATE SKIP LOCKED داشت تا یک پیام دو بار برداشته نشود.

مقایسه

تراکنش	
C#	`using var tx = conn.BeginTransaction(); ... tx.Commit();`
TypeScript	`db.run('BEGIN') ... 'COMMIT' / 'ROLLBACK'`
Python	`with conn:`
Java	`setAutoCommit(false); commit(); rollback();`

C# بدون Commit در Dispose خودش rollback می‌کند؛ sql.js هیچ کمکی ندارد.

backoff	
C#	`Math.Min(10 * Math.Pow(2, n - 1), 1800)`
TypeScript	`Math.min(10 * 2 ** (n - 1), 1800)`
Python	`min(10 * 2 ** (n - 1), 1800)`
Java	`Math.min(10 * Math.pow(2, n - 1), 1800)`

Python و TS عملگر توان (**) دارند؛ C# و Java تابع Pow.

تزریق زمان	
C#	`now` پارامتر / `TimeProvider` (در `ClinicHub`)
TypeScript	`now` پارامتر
Python	`now` پارامتر
Java	`now` پارامتر / `java.time.Clock`

.NET 8 به بعد TimeProvider دارد و FakeTimeProvider برای تست.

کتابخانه‌ی آماده	
C#	`MassTransit` / `Wolverine` outbox
TypeScript	—
Python	—
Java	`Spring Modulith` event publication / `Debezium`

برای یک سرویس، جدول خودتان کافی و روشن‌تر است.

کد

ذخیره‌ی سفارش و پیام با هم

C# کد اجراشده و تست‌شده در /examples — examples/backend/csharp/Portfolio/Outbox.cs

```
public static void PlaceOrder(SqliteConnection conn, string orderId, string patient)
{
    // The order and its message commit together: either both exist or neither does.
    using var tx = conn.BeginTransaction();
    Execute(conn, tx, "INSERT INTO orders (id, patient) VALUES ($a, $b)", orderId, patient);
    Execute(conn, tx, "INSERT INTO outbox (payload) VALUES ($a)", JsonSerializer.Serialize(new { order =
orderId }));
    tx.Commit(); // disposing without Commit rolls back
}
```

TypeScript کد اجراشده و تست‌شده در /examples — examples/backend/typescript/src/outbox.ts

```
export function placeOrder(db: Database, orderId: string, patient: string): void {
  // The order and its message commit together: either both exist or neither does.
  db.run('BEGIN');
  try {
    db.run('INSERT INTO orders (id, patient) VALUES (?, ?)', [orderId, patient]);
    db.run('INSERT INTO outbox (payload) VALUES (?)', [JSON.stringify({ order: orderId })]);
    db.run('COMMIT');
  } catch (err) {
    db.run('ROLLBACK');
    throw err;
  }
}
```

Python کد اجراشده و تست‌شده در /examples — examples/backend/python/portfolio/outbox.py

```
def place_order(conn: sqlite3.Connection, order_id: str, patient: str) -> None:
    # The order and its message commit together: either both exist or neither does.
    with conn:
        conn.execute("INSERT INTO orders (id, patient) VALUES (?, ?)", (order_id, patient))
        conn.execute("INSERT INTO outbox (payload) VALUES (?)", (json.dumps({"order": order_id}),))
```

Java کد اجراشده و تست‌شده در /examples examples/backend/java/src/main/java/portfolio/Outbox.java

```java
public static void placeOrder(Connection conn, String orderId, String patient) throws SQLException {
    // The order and its message commit together: either both exist or neither does.
    conn.setAutoCommit(false);
    try (var order = conn.prepareStatement("INSERT INTO orders (id, patient) VALUES (?, ?)");
         var message = conn.prepareStatement("INSERT INTO outbox (payload) VALUES (?)")) {
        order.setString(1, orderId);
        order.setString(2, patient);
        order.executeUpdate();
        message.setString(1, "{\"order\":\"" + orderId + "\"}");
        message.executeUpdate();
        conn.commit();
    } catch (SQLException e) {
        conn.rollback();
        throw e;
    } finally {
        conn.setAutoCommit(true);
    }
}
```

ارسال با backoff

C# کد اجراشده و تست‌شده در /examples examples/backend/csharp/Portfolio/Outbox.cs

```
public static double BackoffSeconds(int attempts) => Math.Min(10 * Math.Pow(2, attempts - 1), 30 * 60); // 10
s, 20 s, 40 s ... 30 min

public static async Task<int> DispatchAsync(SqliteConnection conn, Func<string, Task> send, double now)
{
    var due = new List<(long Id, string Payload, int Attempts)>();
    using (var cmd = conn.CreateCommand())
    {
        cmd.CommandText = """
            SELECT id, payload, attempts FROM outbox
            WHERE sent IS NULL AND attempts < $max AND (next_attempt IS NULL OR next_attempt <= $now)
            ORDER BY id LIMIT 20
            """;
        cmd.Parameters.AddWithValue("$max", MaxAttempts);
        cmd.Parameters.AddWithValue("$now", now);
        using var r = cmd.ExecuteReader();
        while (r.Read()) due.Add((r.GetInt64(0), r.GetString(1), r.GetInt32(2)));
    }
    var delivered = 0;
    foreach (var (id, payload, prior) in due)
    {
        var attempts = prior + 1;
        try
        {
            await send(payload);
            Execute(conn, null, "UPDATE outbox SET attempts = $a, sent = $b, last_error = NULL WHERE id = $c",
attempts, now, id);
            delivered++;
        }
        catch (Exception ex) // the receiver is down: keep the message, try later
        {
            Execute(conn, null, "UPDATE outbox SET attempts = $a, next_attempt = $b, last_error = $c WHERE id =
$d",
                attempts, now + BackoffSeconds(attempts), ex.Message, id);
        }
    }
    return delivered;
}
```

examples/backend/typescript/src/outbox.ts کد اجراشده و تست‌شده در /examples **TypeScript**

```ts
export const backoffSeconds = (attempts: number) => Math.min(10 * 2 ** (attempts - 1), 30 * 60); // 10 s, 20 s, 40 s ... 30 min

export async function dispatch(db: Database, send: (payload: string) => Promise<void>, now: number):
Promise<number> {
  const due = db.exec(
    `SELECT id, payload, attempts FROM outbox
     WHERE sent IS NULL AND attempts < ? AND (next_attempt IS NULL OR next_attempt <= ?)
     ORDER BY id LIMIT 20`,
    [MAX_ATTEMPTS, now],
  )[0]?.values ?? [];
  let delivered = 0;
  for (const [id, payload, prior] of due as [number, string, number][]) {
    const attempts = prior + 1;
    try {
      await send(payload);
      db.run('UPDATE outbox SET attempts = ?, sent = ?, last_error = NULL WHERE id = ?', [attempts, now, id]);
      delivered++;
    } catch (err) {
      // the receiver is down: keep the message, try later
      db.run('UPDATE outbox SET attempts = ?, next_attempt = ?, last_error = ? WHERE id = ?', [
        attempts, now + backoffSeconds(attempts), (err as Error).message, id,
      ]);
    }
  }
  return delivered;
}
```

examples/backend/python/portfolio/outbox.py کد اجراشده و تست‌شده در /examples **Python**

```python
def backoff_seconds(attempts: int) -> float:
    # 10 s, 20 s, 40 s ... capped at 30 minutes.
    return min(10 * 2 ** (attempts - 1), 30 * 60)


def dispatch(conn: sqlite3.Connection, send: Callable[[str], None], now: float) -> int:
    due = conn.execute(
        """SELECT id, payload, attempts FROM outbox
           WHERE sent IS NULL AND attempts < ? AND (next_attempt IS NULL OR next_attempt <= ?)
           ORDER BY id LIMIT 20""",
        (MAX_ATTEMPTS, now),
    ).fetchall()
    delivered = 0
    for msg_id, payload, attempts in due:
        attempts += 1
        try:
            send(payload)
            conn.execute("UPDATE outbox SET attempts = ?, sent = ?, last_error = NULL WHERE id = ?",
                         (attempts, now, msg_id))
            delivered += 1
        except Exception as e:  # the receiver is down: keep the message, try later
            conn.execute("UPDATE outbox SET attempts = ?, next_attempt = ?, last_error = ? WHERE id = ?",
                         (attempts, now + backoff_seconds(attempts), str(e), msg_id))
        conn.commit()
    return delivered
```

Java کد اجراشده و تست‌شده در /examples examples/backend/java/src/main/java/portfolio/Outbox.java

```
public static double backoffSeconds(int attempts) {
    return Math.min(10 * Math.pow(2, attempts - 1), 30 * 60); // 10 s, 20 s, 40 s ... 30 min
}

record Due(long id, String payload, int attempts) {}

public static int dispatch(Connection conn, Sender sender, double now) throws SQLException {
    var due = new ArrayList<Due>();
    try (var ps = conn.prepareStatement("""
            SELECT id, payload, attempts FROM outbox
            WHERE sent IS NULL AND attempts < ? AND (next_attempt IS NULL OR next_attempt <= ?)
            ORDER BY id LIMIT 20""")) {
        ps.setInt(1, MAX_ATTEMPTS);
        ps.setDouble(2, now);
        try (var rs = ps.executeQuery()) {
            while (rs.next()) due.add(new Due(rs.getLong(1), rs.getString(2), rs.getInt(3)));
        }
    }
    int delivered = 0;
    for (var m : due) {
        int attempts = m.attempts() + 1;
        try {
            sender.send(m.payload());
            try (var ps = conn.prepareStatement("UPDATE outbox SET attempts = ?, sent = ?, last_error = NULL
WHERE id = ?")) {
                ps.setInt(1, attempts);
                ps.setDouble(2, now);
                ps.setLong(3, m.id());
                ps.executeUpdate();
            }
            delivered++;
        } catch (Exception e) { // the receiver is down: keep the message, try later
            try (var ps = conn.prepareStatement("UPDATE outbox SET attempts = ?, next_attempt = ?, last_error =
? WHERE id = ?")) {
                ps.setInt(1, attempts);
                ps.setDouble(2, now + backoffSeconds(attempts));
                ps.setString(3, e.getMessage());
                ps.setLong(4, m.id());
                ps.executeUpdate();
            }
        }
    }
    return delivered;
}
```

اشتباه‌های رایج

همه **ذخیره در دیتابیس و بعد ارسال HTTP در همان درخواست، بدون outbox**

چرا: اگر HTTP شکست بخورد پیام گم می‌شود؛ اگر بعد از HTTP ذخیره شکست بخورد، دو سیستم ناهمگام می‌شوند.

درست: پیام را در همان تراکنش در جدول outbox بگذار.

همه **retry فوری و بی‌پایان**

چرا: سرویسی که تازه بالا آمده زیر سیل retry دوباره می‌افتد؛ پیام خراب تا ابد صف را پر می‌کند.

درست: backoff نمایی با سقف و حداکثر تعداد تلاش.

همه **گیرنده‌ای که تکرار را تحمل نمی‌کند**

چرا: outbox حداقل یک بار تحویل می‌دهد، نه دقیقاً یک بار.

درست: گیرنده با id پیام idempotent باشد.

Java **فراموش کردن setAutoCommit(true) بعد از تراکنش**

چرا: اتصال بعدی‌ها در حالت دستی می‌ماند و تغییرات commit نمی‌شوند.

درست: finally با setAutoCommit(true)، یا TransactionTemplate در Spring.

این کد کجای پروژه‌هاست

`ClinicHub/src/ClinicHub.Api/Endpoints/OrderEndpoints.cs:95`

پیام outbox همراه سفارش در یک SaveChanges

`ClinicHub/src/ClinicHub.Api/Integrations/OutboxDispatcher.cs:25`

DispatchAsync: پیام‌های آماده، تلاش، backoff

`ClinicHub/src/ClinicHub.Api/Integrations/OutboxDispatcher.cs:22`

MaxAttempts = 12

تمرین

▼ 1. تست retry را گام به گام دنبال کن: بعد از تلاش اول، دوم و سوم ناموفق، next_attempt چند است؟

پاسخ: تلاش 1 در 0 → 10. تلاش 2 در 10 → 30. تلاش 3 در 30 → 70.

▼ 2. dispatch را طوری تغییر بده که بعد از MAX_ATTEMPTS سفارش را Failed علامت بزند. کجا و در چه تراکنشی؟

پاسخ: در شاخه‌ی خطا: اگر attempts == MAX_ATTEMPTS، در همان تراکنشِ آپدیت پیام، وضعیت سفارش را Failed کن. ClinicHub همین کار را می‌کند.

▼ 3. اگر دو نمونه از سرویس هم‌زمان dispatch را اجرا کنند چه می‌شود؟ یک راه حل در SQL Server بگو.

پاسخ: هر دو همان پیام‌ها را برمی‌دارند و دو بار می‌فرستند. راه حل: SELECT TOP 20 ... WITH (UPDLOCK, READPAST)، یا آپدیت شرطی که پیام را به نام یک نمونه «رزرو» کند.

جمله‌ی مصاحبه

The message is saved in the same transaction as the data and delivered later with capped exponential backoff, so a down receiver never loses it.

درس 12 · backend

Calling an LLM with tools

فراخوانی مدل زبانی با ابزار: حلقه، سقف، و ابزار فقط‌خواندنی

claude api | tool use | llm

بعد از این درس می‌توانی

- حلقه‌ی tool use را بنویسی: مدل ابزار می‌خواهد، تو اجرا می‌کنی، نتیجه را برمی‌گردانی، تا جواب نهایی.
- حلقه را محدود کنی: سقف نوبت، رسیدگی به refusal و max_tokens، ابزار شکست‌خورده با is_error.
- ابزارها را فقط‌خواندنی طراحی کنی تا نتیجه‌ی آلوده (prompt injection) نتواند کاری انجام دهد.

سناریو از پروژه‌ی واقعی

جعبه‌ی «Ask the portfolio» در Control Panel سؤال را به core-api می‌فرستد. core-api با Claude یک حلقه اجرا می‌کند و سه ابزار فقط‌خواندنی دارد: فهرست پروژه‌ها، وضعیت یک پروژه، و تحلیل جست‌وجوی کار از LedgerDashboard. حداکثر ۶ نوبت، ۲۰ سؤال در ساعت برای هر کاربر، همه‌ی نتیجه‌های ابزار در یک پیام، و fallbacks برای درخواست‌های ردشده. هیچ ابزاری نمی‌نویسد، SQL اجرا نمی‌کند یا URL نمی‌گیرد. این درس همان حلقه را در چهار زبان با SDK رسمی Anthropic می‌سازد.

سؤال چرا همه‌ی tool_resultها در یک پیام user؟

پاسخ مدل ممکن است چند ابزار را هم‌زمان بخواهد. اگر نتیجه‌ها را در چند پیام جدا بفرستی، مدل یاد می‌گیرد که دیگر فراخوانی موازی نکند. یک پیام با همه‌ی نتیجه‌ها شکل درست است.

سؤال اگر ابزار خطا بدهد، نتیجه را حذف کنیم؟

پاسخ نه. هر tool_use باید tool_result داشته باشد، وگرنه درخواست بعدی رد می‌شود. خطا را با is_error: true برگردان تا مدل بداند و راه دیگری امتحان کند.

مفهوم‌ها

حلقه

درخواست → اگر stop_reason == tool_use: کل پاسخ مدل را بدون تغییر به messages اضافه کن، ابزارها را اجرا کن، نتیجه‌ها را در یک پیام user برگردان، تکرار. اگر end_turn: متن را برگردان.

stop_reasonها

end_turn: جواب تمام شد. tool_use: ابزار می‌خواهد. max_tokens: جواب نصفه ماند. refusal: مدل درخواست را رد کرد. قبل از خواندن content همیشه stop_reason را چک کن.

مدل و پارامترها

claude-opus-5-5 با output_config.effort = medium. thinking روی این مدل همیشه روشن است و نباید غیرفعالش کرد؛ عمق را با effort تنظیم کن. fallbacks با beta server-side-fallback در Python و TypeScript و C# روشن است: اگر درخواست رد شود، در همان فراخوانی روی مدل جایگزین اجرا می‌شود.

C# و Java: بدون ToParam در C#

SDK پایتون و TypeScript اجازه می‌دهند response.content را مستقیم برگردانی. در C# باید هر بلوک را به نوع Param متناظرش تبدیل کنی (Text، Thinking با Signature، ToolUse). Java یک response.toParam() دارد.

ابزار فقط‌خواندنی

نتیجه‌ی ابزار داده است، نه دستور. ولی اگر ابزاری بتواند بنویسد، متنی در داده می‌تواند مدل را قانع کند که بنویسد. بهترین دفاع این است که چنین ابزاری وجود نداشته باشد: lookup ثابت، بدون SQL، بدون URL.

تست بدون هزینه

Python و TypeScript تابع create را تزریق می‌کنند و در تست پاسخ جعلی می‌دهند. C# و Java کلاینت واقعی SDK را به یک سرور HTTP محلی وصل می‌کنند که JSON آماده برمی‌گرداند؛ پس کد SDK واقعاً اجرا می‌شود، ولی هیچ درخواستی به Anthropic نمی‌رود.

مقایسه

کلاینت	
C#	`new AnthropicClient()`
TypeScript	`new Anthropic()`
Python	`anthropic.Anthropic()`
Java	`AnthropicOkHttpClient.fromEnv()`

همه ANTHROPIC_API_KEY را از محیط می‌خوانند.

درخواست با ابزار

C#	`client.Beta.Messages.Create(new MessageCreateParams { Tools = [...] })`
TypeScript	`client.beta.messages.create({ tools: [...] })`
Python	`client.beta.messages.create(tools=[...])`
Java	`client.messages().create(MessageCreateParams.builder().addTool(tool)...)`

Java از builder استفاده می‌کند؛ نمونه‌ی Java ما beta و fallbacks ندارد چون شکل آن در مستندات SDK جاوا ثبت نشده.

برگرداندن نوبت مدل

C#	`*BlockParam تبدیل دستی هر بلوک به`
TypeScript	`{ role: 'assistant', content: response.content }`
Python	`{"role": "assistant", "content": response.content}`
Java	`messages.add(response.toParam())`

C# تنها جایی است که باید بلوک به بلوک تبدیل کنی.

نتیجه‌ی ابزار با خطا

C#	`new BetaToolResultBlockParam { IsError = true }`
TypeScript	`{ type: 'tool_result', is_error: true }`
Python	`{"type": "tool_result", "is_error": True}`
Java	`ToolResultBlockParam.builder().isError(true)`

یکسان؛ فقط نام‌گذاری زبان.

fallbacks	
C#	`Fallbacks = new BetaFallbacksParam([...])`
TypeScript	`fallbacks: 'default'`
Python	`fallbacks="default"`
Java	در نمونه نیست

C# SDK نسخه‌ی 13 شکل لیستی را می‌پذیرد (beta 2026-06-01)؛ Python و TS شکل default را (beta 2026-07-01).

کد

یک فراخوانی

C# کد اجراشده و تست‌شده در /examples `examples/backend/csharp/Portfolio/Assistant.cs`

```
private static MessageCreateParams Request(List<BetaMessageParam> messages) => new()
{
    Model = "claude-opus-5-5",
    MaxTokens = 16000,
    System = SystemPrompt,
    Tools =
    [
        new BetaTool
        {
            Name = "get_project_status",
            Description = "Get one project's live status by its key, e.g. 'ledgerdash'.",
            InputSchema = new()
            {
                Properties = new Dictionary<string, JsonElement>
                {
                    ["key"] = JsonSerializer.SerializeToElement(new { type = "string" }),
                },
                Required = ["key"],
            },
        },
    ],
    Messages = messages,
    OutputConfig = new BetaOutputConfig { Effort = Effort.Medium },
    // A declined request is re-run on the fallback model in the same call.
    Betas = ["server-side-fallback-2026-06-01"],
    Fallbacks = new BetaFallbacksParam([new BetaFallbackParam(NonBeta.Model.ClaudeOpus4_8)]),
};
```

TypeScript کد اجراشده و تست‌شده در /examples

examples/backend/typescript/src/assistant.ts

```typescript
export function callClaude(client: Anthropic, messages: MessageParam[]): Promise<Message> {
  return client.beta.messages.create({
    model: MODEL,
    max_tokens: 16000,
    system: SYSTEM_PROMPT,
    tools: TOOLS,
    messages,
    output_config: { effort: 'medium' },
    // A declined request is re-run on Anthropic's recommended fallback model.
    betas: ['server-side-fallback-2026-07-01'],
    fallbacks: 'default',
  });
}
```

Python کد اجراشده و تست‌شده در /examples

examples/backend/python/portfolio/assistant.py

```python
def call_claude(client: anthropic.Anthropic, messages: list) -> Any:
    return client.beta.messages.create(
        model=MODEL,
        max_tokens=16000,
        system=SYSTEM_PROMPT,
        tools=TOOLS,
        messages=messages,
        output_config={"effort": "medium"},
        # A declined request is re-run on Anthropic's recommended fallback model.
        betas=["server-side-fallback-2026-07-01"],
        fallbacks="default",
    )
```

Java کد اجراشده و تست‌شده در /examples

examples/backend/java/src/main/java/portfolio/Assistant.java

```java
private static MessageCreateParams request(List<MessageParam> messages) {
    return MessageCreateParams.builder()
            .model("claude-opus-5-5")
            .maxTokens(16000L)
            .system(SYSTEM_PROMPT)
            .addTool(TOOL)
            .messages(messages)
            .outputConfig(OutputConfig.builder().effort(OutputConfig.Effort.MEDIUM).build())
            .build();
}
```

حلقه‌ی ابزار

C# کد اجراشده و تست‌شده در /examples examples/backend/csharp/Portfolio/Assistant.cs

```
public static async Task<string> AskAsync(AnthropicClient client, Func<string, JsonElement, Task<string>>
runTool, string question)
{
    List<BetaMessageParam> messages = [new BetaUserMessageParam { Content = question }];
    for (var turn = 0; turn < MaxTurns; turn++)
    {
        BetaMessage response = await client.Beta.Messages.Create(Request(messages));
        if (response.StopReason == "refusal") return "I can't help with that question.";
        if (response.StopReason == "max_tokens") throw new InvalidOperationException("The answer was cut
off.");
        if (response.StopReason != "tool_use")
            return string.Join("\n", response.Content.Select(b => b.TryPickText(out var t) ? t.Text :
null).OfType<string>()).Trim();

        // C# has no ToParam(): the assistant turn is rebuilt block by block and sent back unchanged.
        List<BetaContentBlockParam> echo = [];
        List<BetaContentBlockParam> results = [];
        foreach (var block in response.Content)
        {
            if (block.TryPickText(out var text)) echo.Add(new BetaTextBlockParam { Text = text.Text });
            else if (block.TryPickThinking(out var thinking))
                echo.Add(new BetaThinkingBlockParam { Thinking = thinking.Thinking, Signature =
thinking.Signature });
            else if (block.TryPickToolUse(out var call))
            {
                echo.Add(new BetaToolUseBlockParam { ID = call.ID, Name = call.Name, Input = call.Input });
                try
                {
                    results.Add(new BetaToolResultBlockParam { ToolUseID = call.ID, Content = await
runTool(call.Name, JsonSerializer.SerializeToElement(call.Input)) });
                }
                catch (Exception ex) // a failed tool is reported to the model, not dropped
                {
                    results.Add(new BetaToolResultBlockParam { ToolUseID = call.ID, Content = ex.Message,
IsError = true });
                }
            }
        }
        // All tool results go back in ONE user message.
        messages.Add(new BetaAssistantMessageParam { Content = echo });
        messages.Add(new BetaUserMessageParam { Content = results });
    }
    return $"I couldn't finish within {MaxTurns} steps.";
}
```

examples/backend/typescript/src/assistant.ts کد اجراشده و تست‌شده در /examples **TypeScript**

```ts
export async function ask(
  create: (messages: MessageParam[]) => Promise<Message>,
  runTool: (name: string, input: Record<string, unknown>) => Promise<string>,
  question: string,
): Promise<string> {
  const messages: MessageParam[] = [{ role: 'user', content: question }];
  for (let turn = 0; turn < MAX_TURNS; turn++) {
    const response = await create(messages);
    if (response.stop_reason === 'refusal') return "I can't help with that question.";
    if (response.stop_reason === 'max_tokens') throw new Error('The answer was cut off.');
    if (response.stop_reason !== 'tool_use') {
      return response.content
        .flatMap((b) => (b.type === 'text' ? [b.text] : []))
        .join('\n')
        .trim();
    }
    // The whole assistant turn goes back unchanged; all tool results come back in ONE user message.
    messages.push({ role: 'assistant', content: response.content });
    const calls = response.content.filter((b): b is Anthropic.Beta.Messages.BetaToolUseBlock => b.type ===
'tool_use');
    const results = await Promise.all(
      calls.map(async (call): Promise<ToolResult> => {
        try {
          return { type: 'tool_result', tool_use_id: call.id, content: await runTool(call.name, call.input as
Record<string, unknown>) };
        } catch (err) {
          // a failed tool is reported to the model, not dropped
          return { type: 'tool_result', tool_use_id: call.id, content: (err as Error).message, is_error: true
};
        }
      }),
    );
    messages.push({ role: 'user', content: results });
  }
  return `I couldn't finish within ${MAX_TURNS} steps.`;
}
```

examples/backend/python/portfolio/assistant.py کد اجراشده و تست‌شده در /examples **Python**

```python
def ask(create: Callable[[list], Any], run_tool: Callable[[str, dict], str], question: str) -> str:
    messages: list = [{"role": "user", "content": question}]
    for _ in range(MAX_TURNS):
        response = create(messages)
        if response.stop_reason == "refusal":
            return "I can't help with that question."
        if response.stop_reason == "max_tokens":
            raise RuntimeError("The answer was cut off.")
        if response.stop_reason != "tool_use":
            return "\n".join(b.text for b in response.content if b.type == "text").strip()

        # The whole assistant turn goes back unchanged; all tool results come back in ONE user message.
        messages.append({"role": "assistant", "content": response.content})
        results = []
        for call in (b for b in response.content if b.type == "tool_use"):
            try:
                content = run_tool(call.name, call.input)
                results.append({"type": "tool_result", "tool_use_id": call.id, "content": content})
            except Exception as e:  # a failed tool is reported to the model, not dropped
                results.append({"type": "tool_result", "tool_use_id": call.id, "content": str(e), "is_error":
True})
        messages.append({"role": "user", "content": results})
    return f"I couldn't finish within {MAX_TURNS} steps."
```

Java کد اجراشده و تست‌شده در examples/

examples/backend/java/src/main/java/portfolio/Assistant.java

```
public static String ask(AnthropicClient client, BiFunction<String, JsonValue, String> runTool, String
question) {
    var messages = new ArrayList<MessageParam>();
    messages.add(MessageParam.builder().role(MessageParam.Role.USER).content(question).build());
    for (int turn = 0; turn < MAX_TURNS; turn++) {
        Message response = client.messages().create(request(messages));
        var stop = response.stopReason().orElse(StopReason.END_TURN);
        if (stop.equals(StopReason.REFUSAL)) return "I can't help with that question.";
        if (stop.equals(StopReason.MAX_TOKENS)) throw new IllegalStateException("The answer was cut off.");
        if (!stop.equals(StopReason.TOOL_USE)) {
            return response.content().stream()
                    .flatMap(b -> b.text().stream())
                    .map(t -> t.text())
                    .collect(Collectors.joining("\n")).trim();
        }
        // The whole assistant turn goes back unchanged; all tool results come back in ONE user message.
        messages.add(response.toParam());
        var results = new ArrayList<ContentBlockParam>();
        for (var block : response.content()) {
            block.toolUse().ifPresent(call -> {
                var result = ToolResultBlockParam.builder().toolUseId(call.id());
                try {
                    result.content(runTool.apply(call.name(), call._input()));
                } catch (RuntimeException e) { // a failed tool is reported to the model, not dropped
                    result.content(e.getMessage()).isError(true);
                }
                results.add(ContentBlockParam.ofToolResult(result.build()));
            });
        }

messages.add(MessageParam.builder().role(MessageParam.Role.USER).contentOfBlockParams(results).build());
    }
    return "I couldn't finish within " + MAX_TURNS + " steps.";
}
```

اشتباه‌های رایج

همه **حلقه‌ی بدون سقف**

چرا: مدل ممکن است بارها ابزار بخواهد؛ هزینه و زمان بی‌حد می‌شود.

درست: MAX_TURNS، و محدودیت تعداد سؤال برای هر کاربر.

همه **خواندن response.content بدون چک stop_reason**

چرا: در refusal یا max_tokens محتوا ناقص یا خالی است.

درست: اول stop_reason را چک کن.

همه **دادن ابزار نوشتن یا اجرای SQL به دستیار**

چرا: یک رشته در داده (نام شرکت، ایمیل، خطای adapter) می‌تواند مدل را به اجرای کار ناخواسته قانع کند (prompt injection).

درست: ابزار فقط‌خواندنی با ورودی محدود (enum، کلید)؛ کار نوشتن را به UI معمولی بسپار.

C# **new BetaContentBlockParam(block.Json) برای برگرداندن نوبت مدل**

چرا: کامپایل می‌شود ولی مقدار typed آن null است و TryPick کار نمی‌کند.

درست: هر بلوک را به BetaTextBlockParam، BetaThinkingBlockParam (با Signature) و BetaToolUseBlockParam تبدیل کن.

همه **غیرفعال کردن thinking روی claude-opus-5-5**

چرا: این مدل thinking را خاموش نمی‌کند و درخواست 400 می‌گیرد.

درست: thinking را نفرست و effort را پایین بیاور (low/medium).

این کد کجای پروژه‌هاست

`ControlPanel/services/core-api/src/assistant/assistant.service.ts:35`

MAX_TURNS = 6

`ControlPanel/services/core-api/src/assistant/assistant.service.ts:95`

همه‌ی tool_resultها با هم، خطا با is_error

`ControlPanel/services/core-api/src/assistant/assistant.service.ts:124`

'fallbacks: 'default

`ControlPanel/services/core-api/src/assistant/assistant.tools.ts:1`

سه ابزار فقط‌خواندنی

تمرین

▼ 1. در تست TypeScript، نتیجه‌ی ابزار دوم را در یک پیام جدا بفرست (دو پیام user). کدام assert شکست می‌خورد، و API واقعی چه می‌کرد؟

پاسخ: assert روی طول (2) results شکست می‌خورد. API واقعی درخواست را قبول می‌کند ولی مدل را به فراخوانی‌های سریالی عادت می‌دهد؛ اگر tool_use بدون نتیجه بماند 400 می‌دهد.

▼ 2. یک ابزار دوم list_projects اضافه کن (بدون ورودی). چه چیزهایی در هر زبان عوض می‌شود؟

پاسخ: تعریف ابزار در TOOLS و شاخه‌ی جدید در portfolioTool / run_tool. خود حلقه هیچ تغییری نمی‌کند.

▼ 3. چرا ابزار get_project_status کلید پروژه را می‌گیرد و نه URL adapter را؟

پاسخ: اگر URL بگیرد، مدل (یا متنی در داده) می‌تواند آن را به آدرس داخلی دیگری اشاره دهد (SSRF). با کلید، فقط پروژه‌های ثبت‌شده در دسترس‌اند.

جمله‌ی مصاحبه

My assistant runs a bounded tool loop with read-only tools, returns every tool result in one message, and checks the stop reason before reading the answer.